



School of Computing

College of Engineering, Computing
and Cybernetics (CECC)

ILP-Based Delete Relaxation Heuristics for Partial Order Causal Link (POCL) Planning

— 12 pt research project (S2/S1 2023–2024)

A report submitted for the course
COMP3770, Computing Research Project (R&D)

By:
Scott Howsam

Supervisor:
Dr. Pascal Bercher

May 2024

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

May, Scott Howsam

Abstract

Partial Order Causal Link (POCL) planning uses partial plans as search nodes to maintain a principle of least commitment, allowing each node to represent up to a factorial number of action sequences. POCL nodes also store constraints on these possibilities using causal links, which provide complex plan information by committing to causal relationships between actions. These information-dense nodes provide a notional advantage for solving plan existence, as pruning a single infeasible POCL node is equivalent to pruning up to a factorial number of classical state-based search nodes. Yet current state-of-the-art POCL heuristics fail to capitalise on this, ignoring most of the information within search nodes in order to create heuristics that run in polynomial time. Progress in search is represented by the content of search nodes, and erasing this current progress to achieve further progress is considered inconceivable in other planning domains, yet it is the norm in POCL planning. By contrast, we propose to maintain this important information. We introduce new heuristics built with integer programming models which accurately translate classical delete-relaxation heuristics to POCL planning by maintaining critical search node information. This significantly reduces the search space, at the cost of slower calculation due to these heuristics being NP-hard to compute.

Table of Contents

1	Introduction	1
2	Background	5
2.1	Formalisation of Propositional STRIPS Planning	5
2.2	Background on LPs and ILPs	11
2.3	Formalisation of LPs and ILPs	12
3	Related Work	15
3.1	ILPs in Planning	15
3.2	POCL Heuristics	17
4	Well-Informed Delete Relaxation Heuristics for POCL Planning	19
4.1	ILP Formulation of a Delete Relaxed POCL Problem	19
4.2	ILP Formulation of a Counting-Based Relaxation	26
5	Evaluation	33
5.1	Evaluated Software	33
5.2	Benchmarks and Setup	34
5.3	Empirical Results	35
6	Future Work	43
7	Conclusion	45
	Bibliography	47

Introduction

Partial Order Causal Link (POCL) planning is a solving technique in planning which maintains the principle of least commitment, as explained by [Weld \(1994\)](#). While state-based planning approaches search the space of states, requiring commitment to a specific ordered sequence of actions which reaches that state, POCL planning searches the space of partial plans, where actions only have a partial set of ordering constraints, and so no such commitments need to be made. As explained by [Minton et al. \(1994\)](#), a single partial plan can hence represent up to a factorial number of action sequences, and hence POCL search trees can be exponentially smaller than their total-order counterparts, which [Weld \(2011\)](#) notes was used as a justification for the intuitive superiority of partial-order planning until the mid 90s. However, [Minton et al. \(1994\)](#) also found that this assumption of superiority required question, since ‘the superiority of partial-order planning *can* depend critically upon the search strategy and search heuristics employed’. Total-order state based planning allows for intuitive, easy to compute, and powerful heuristics and search strategies, whereas partial-order planning struggled to generate any comparable heuristics, and this began to shift planning research toward total-order planning. Part of this shift was the development of Graphplan by [Blum and Furst \(1995\)](#), which took a fundamentally different approach to traditional partial-order planners, and outperformed UCPOP, the leading partial-order planner of the time created by [Penberthy and Weld \(1992\)](#). This shift continued with UNPOP, a state-based solver created by [McDermott \(1996\)](#) which automatically generated domain-independent heuristics that allowed total-order state-based planning, which Graphplan had not fully embraced as it still produced partial-order outputs, to become competitive with the best planners of the time.

Today, state-based planning is still the dominant technique, with the FF planning system designed by [Hoffmann and Nebel \(2001\)](#), and then the Fast Downward planning system designed by [Helmert \(2006\)](#), significantly outperforming partial-order planners of the

1 Introduction

time such as VHPOP designed by [Younes and Simmons \(2003\)](#). However, attempts at proving that the complexity of partial-order planning is higher than total-order planning has yielded fewer results. Both partial-order planning and total-order planning are PSPACE-complete problems, even in the simplest form as proven by [Bylander \(1994\)](#), with [Bercher \(2021\)](#) confirming that this is also the case for POCL planning, and so any attempt to differentiate their complexity must investigate differences with the PSPACE-complete complexity class. Using sub-classes of PSPACE-completeness, such as those proposed by [Backstrom and Jonsson \(2011\)](#), still does not find partial-order planning to be higher in complexity than total-order planning. Instead, it is possible that heuristic efficiency could be at fault for the current poor performance of partial-order planners, a possibility which [Balcázar \(1996\)](#) notes is important for all heuristic search algorithms on graphs. [Bercher \(2021\)](#) finds that a common heuristic, the delete-relaxation heuristic, is NP-complete in POCL planning, compared to polytime for state-based planning, which provides further evidence for heuristic effectiveness and efficiency being a key drawback of POCL planning.

The input for heuristics in partial-order planning is more complex than state-based planning. In state-based planning, the current search progress can be fed to a heuristic algorithm as a constant size list of variables and their values. In POCL planning, this search progress contains a list of steps, which can contain repeated actions, a set of orderings between those steps, which can grow quadratically with the number of steps, and a set of causal links between effects and preconditions of these steps, which grows linearly with the number of steps. In addition, the size of each search node grows with search depth in POCL planning as more steps, orderings, and causal links are added, whereas it remains constant in state-based planning as the total number of existing propositional variables never changes. This problem has been recognised when creating heuristics for partial-order planning, and the two major well-informed domain-independent heuristics for POCL planning, the *Relax heuristic* created by [Nguyen and Kambhampati \(2001\)](#), and the *Additive heuristic* created by [Younes and Simmons \(2003\)](#), solve it by including limited amounts of partial-plan information within their input. The additive heuristic ignores causal links and delete effects of actions within the current partial-plan, and the relax heuristic additionally ignores ordering constraints between actions, but doesn't make the assumption of subgoal independence, accounting for positive interactions between subgoals similarly to the FF heuristic by [Hoffmann and Nebel \(2001\)](#). This reduction of search node components to consider when generating heuristic estimates allows both of these heuristics to run in polytime.

The recognition of heuristic input complexity having a significant effect on planning efficiency causes a major issue in POCL planning. While ignoring most partial-plan information when calculating heuristics has led to the creation of informed heuristics that are very fast to compute, such as the Relax and Additive heuristics, these heuristics are significantly less informed than if they accounted for all of the information within the

partial-plan they are calculated on. Causal links in particular contain a large amount of information, as they commit to causal relationships between actions, specifying that an effect of one action is required to protect the precondition of a later action, preventing actions that delete the linked proposition from being inserted between the two linked actions. This induces a constraint on where actions can be included within a plan, and hence implicitly prunes a large number of possible action sequences. The delete effects of actions within a partial plan also have a pruning effect, as they delete propositions which could have otherwise been used as preconditions for actions in places where those actions realistically couldn't appear if delete effects were included. Hence, causal links and delete effects encode a large amount of the search progress to reach a certain node, and by ignoring them within a heuristic, that heuristic is essentially being calculated for a significantly more abstract partial plan. This abstraction might be similar to going back many layers of depth in the search tree, and could be equivalent to many other explored search nodes, meaning that that these heuristics cannot differentiate between important differences in search nodes. In any other planning paradigm, running such a heuristic would likely be considered absurd, due to the fact that it is sacrificing large amounts of already computed search progress in order to save computation time in further expansion of the search tree.

We present new informed and admissible heuristics which take into account these often ignored components of a partial-plan search node. It has been shown by [Bercher \(2021\)](#) that a delete-relaxed heuristic which respects ordering constraints; as well as either current causal links, delete effects, or both; will have an underlying decision problem which is NP-complete. However, as established previously, each POCL node can represent up to a factorial amount of action sequences, and so it is very possible that having a much more informed heuristic will unlock much of these benefits to partial-order planning that previous heuristics couldn't. Hence, an NP-complete heuristic might turn out to be worthwhile given the benefits to the search process. These heuristics are particularly promising in their application to satisfiability: determining whether a planning problem instance is solvable. State-based planners can struggle when presented with infeasible planning problems, as their smaller amount of information per search node compared to POCL planning can lead to a search tree that grows faster than infeasible branches can be pruned. This is also the case with the Relax and Additive heuristics for POCL planning, since they don't consider the large amount of additional information available within POCL search nodes. We propose that an informed POCL heuristic that takes into account all details of the much more complex structure of a POCL node could determine infeasibility for factorial numbers of action sequences in a single NP-complete calculation, possibly allowing it to prune branches faster than the search tree can grow in cases where state-based planning cannot, providing a potential benefit for determining infeasibility in planning.

Background

This chapter introduces the theoretical background for our heuristics. We first introduce a formal framework for planning which will be used for the remainder of this report. We then introduce and provide a framework for Integer Linear Programs, explaining how they can be used to solve relaxed POCL planning problems for heuristic estimates.

2.1 Formalisation of Propositional STRIPS Planning

Here we provide the formal framework for planning which will be used for the remainder of this report. We consider the problem of determining solvability and minimising actions for partially ordered plans. This framework is built from the language of *Propositional STRIPS* planning. *Propositional* means that each variable is a proposition, signifying it can only be true or false. *STRIPS* is a planning notation defined by [Fikes and Nilsson \(1971\)](#). We first define a classical planning problem, which is usually used in *state-based* planning, where search nodes are represented by the set of propositions and their values, called a *state*, along with a sequence of actions which lead from the initial state to the current state. We then expand these definitions to work with *Partial Order Causal Link* (POCL) planning, where each search node is its own *partial plan*. These partial plans contain action instances called *steps*, orderings between those steps, and causal links representing causal relationships between effects and preconditions of different steps. We follow the notation used by [Bercher and Olz \(2020\)](#).

A *classical planning problem* $\mathcal{P}$ is given by a tuple (V, A, s_I, g) , where V is a finite set of propositional state variables, A is a finite set of *actions*, $s_I \in 2^V$ is the initial state, and $g \subseteq V$ is the goal description. An *action* $a \in A$ is a tuple (pre, add, del) ; referred to as $pre(a)$, $add(a)$, and $del(a)$; where $pre, add, del \subseteq V$. These sets represent an action's

2 Background

precondition, *add*, and *delete* lists. The precondition list holds the propositions required to execute an action, and so $a \in A$ is executable in a state $s \in 2^V$ if and only if $pre(a) \subseteq s$. The add list holds the propositions that will be set to true after an action is executed, and the delete lists similarly holds the propositions which will be set to false. From this, if a is executable in s , we can define a state transition function $\gamma : A \times 2^V \rightarrow 2^V$ to return the successor state after executing a in s : $\gamma(a, s) = (s \setminus del(a)) \cup add(a)$. A sequence of actions $\bar{a} = (a_0, a_1, \dots, a_n)$ is executable in a state s_0 if there exists a state sequence $\bar{s} = (s_1, s_2, \dots, s_{n+1})$ such that each successive action in $\bar{a}$ transitions to the next state in $\bar{s}$, or formally that $\gamma(a_{i-1}, s_{i-1}) = s_i$ holds for all $1 \leq i \leq n+1$. The state s_{n+1} is called the *result* of executing the sequence. In such a sequence, any proposition where $v \in s_{i+1}$ but $v \notin s_i$ is said to be *achieved* by the action a_i . If a state $s \in 2^V$ is such that $g \subseteq s$ then it is called a *goal state*.

Classical solutions to planning problems are called *total-order* plans, since they are defined with action sequences which follow a sequential order, as defined below. Figure 2.1 shows an example of a total-order plan.

Definition 1. An action sequence $\bar{a}$ is called a *total-order plan* $\bar{P}$ (also *total-order solution*) for a classical planning problem if and only if it is executable in s_I and results in a goal state.

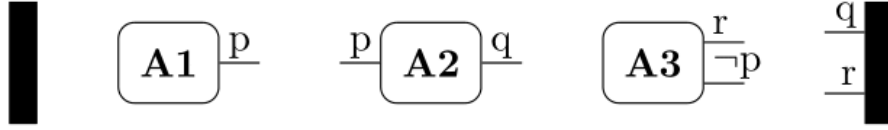


Figure 2.1: An example of a total-order plan. Actions are totally ordered from left to right.

For the creation of our heuristic, we emulate the delete relaxation heuristics of state-based total-order planning. These heuristics utilise the delete relaxation of classical planning problems. In state-based planning, a search node represents a state, and so by substituting s_I for the state of the current search node, a new classical planning problem can be created, and the solution length of its delete relaxation can be used as a heuristic value. Hence, it is important to more formally define the delete relaxation of a classical problem..

The *delete relaxation* of a classical planning problem $\mathcal{P}$ is the problem $\mathcal{P}^+$, which is $\mathcal{P}$ but with $del(a) = \emptyset$ for all $a \in A$, meaning that all delete effects of actions are removed. By ignoring all delete effects in future actions, any action that becomes executable will forever remain executable. This advantage makes $\mathcal{P}^+$ a polytime problem, as shown by Bylander (1994).

2.1 Formalisation of Propositional STRIPS Planning

We will next extend this definition to POCL plans. POCL planning does not produce total-order plans, and instead relies on *orderings* and *causal links* to produce *partially ordered* plans with up to a factorial number of *linearizations*: total-order plans which meet the ordering and causality criteria of the POCL plan. When using POCL search algorithms to solve planning problems, search nodes are *partial plans*, as opposed to classical planning where each search node is a *state*. We will also refer to these partial plans as *plans* for short when it is clear from context that we are referring to a partial plan and not a solution.

A *partial plan* P is given by a tuple $(PS, \prec, CL)$, where PS is a finite set of *plan steps*, $ps = (l, a)$, with l being a unique label and $a \in A$ being an action, $\prec$ is a *partial order* on PS , and CL is a finite set of *causal links*. Plan steps require unique labels to differentiate between multiple occurrences of the same action within a partial plan. As with actions, we use $pre(ps)$, $add(ps)$, and $del(ps)$ to refer to the precondition, add, and delete lists of a plan step's action. A partial order is a finite set of orderings between steps, where $ps_n \prec ps_m$ represents that the step n must precede the step m in any linearization of the plan. A causal link is used to formally represent a causal dependency between plan steps, and is given by a tuple $(ps_p, v, ps_c) \in CL$ where $ps_p \in PS, v \in V, ps_c \in PS$. A causal link $cl = (ps_p, v, ps_c) \in CL$ requires that v is both a precondition of ps_c , which is the *consumer* of the causal link cl , and an add effect of ps_p , which is the *producer* of cl . We also say that the variable v is *protected* by the causal link cl , as the causal link ensures that v will never be deleted between the producer step ps_p and the consumer step ps_c . Any plan step ps_t which has the potential to violate the protection of the causal link cl , by having both $v \in del(ps_t)$ and the potential to be ordered between ps_p and ps_c , is called a *threatening* plan step to cl . The partial plan P raises a *causal threat* whenever such a threatening plan step exists, denoting a *flaw* in the plan that must be *resolved* to uphold the protection of the threatened causal link. This causal threat can be resolved by ensuring that the threatening step ps_t is either ordered before ps_p , or after ps_c , by adding an ordering constraint. We always assume for simplicity and without loss of generality that there is an implicit ordering constraint $ps_p \prec ps_c$ created for each causal link.

In POCL planning, the problem's initial state and goal description are encoded using two artificial actions, labelled *init* and *goal*, where $add(init) = s_I$, and $pre(goal) = g$. The sets $pre(init)$, $del(init)$, $add(goal)$, and $del(goal)$ are all empty. In all partial plans, *init* is ordered before all other steps and *goal* is ordered after them. This ensures that the only role of *init* is to establish all initial propositions at the beginning of the plan, and the only role of *goal* is to ensure that the goal description is met at the end of the plan. Having these artificial actions allows all reasoning to be done with actions, removing all state-based logic from POCL planning.

2 Background

We can now define POCL solutions to classical planning problems. Figure 2.2 shows an example of a POCL plan, for which the total-order plan from Figure 2.1 is a linearization. This partial plan has a causal threat, as A3 is threatening the causal link between A1 and A2. Figure 2.3 resolves this threat by adding an ordering constraint.

Definition 2. A partial POCL plan P is called a *POCL plan* (also *POCL solution*) for a classical planning problem if and only if every precondition is supported by a causal link and there are no causal threats.

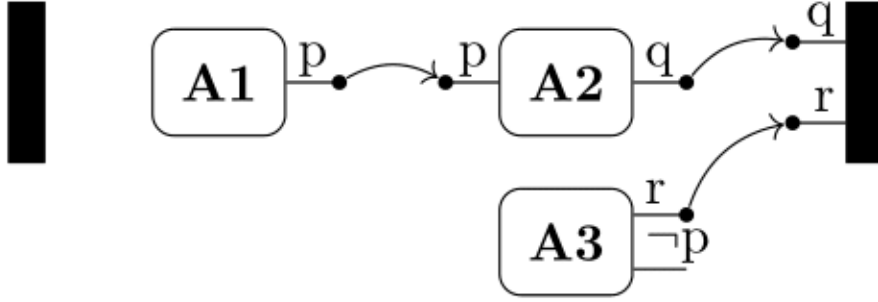


Figure 2.2: An example of a partial-order plan with a threat.

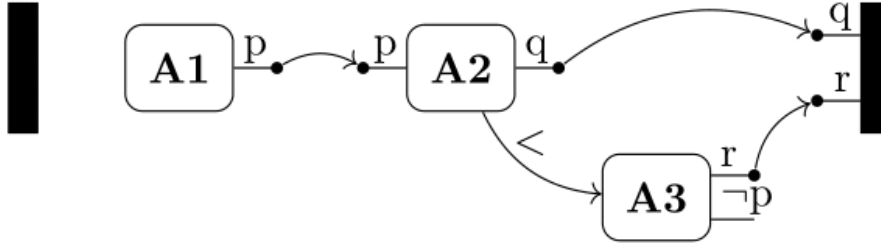


Figure 2.3: An example of a partial-order plan where orderings have resolved threats.

It is common knowledge that these criteria for a POCL plan imply that every linearization of its steps is executable, and hence a POCL plan can represent many total-order solutions. However, note that the inverse is not true. There exist partial-order plans where every linearization is executable, yet there is no way to insert causal links to support every precondition without creating a causal threat which then requires the insertion of another ordering constraint, and hence there is no corresponding POCL plan with the same set of linearizations. This property is explored by [Bercher and Olz \(2020\)](#), with Figure 1 of their paper including such a counterexample. The paper also presents a polytime method for converting any partial-order plan where every linearization is executable into a POCL plan with the same makespan.

2.1 Formalisation of Propositional STRIPS Planning

It is also well known that there is a correspondence between total-order solutions and POCL solutions to classical planning problems.

Proposition 1. *Let $\mathcal{P}$ be a classical planning problem. There is a POCL solution to $\mathcal{P}$ if and only if there is a total-order solution to $\mathcal{P}$.*

The forward implication of this proposition is clear from the fact that a POCL solution represents multiple linearizations, each of which is a valid total-order solution due to following the partial-order constraints. For example, the total-order solution in Figure 2.1 is one possible linearization of a POCL solution shown in Figure 2.2. The backward direction can be done in polytime by inserting required causal links into a total-order plan to create a POCL plan which still maintains total ordering, as this is a permissible special case of partial ordering. This method is described by Muise et al. (2016) in Section 3.2.1.

In order to define POCL heuristics, we need to define a planning problem in terms of the current search node. For state-based planning this is simple to do within the framework of classical planning, as s_I just needs to be updated to the current state of the search node. For POCL planning this becomes more difficult, as search nodes are partial plans containing orderings and causal links, neither of which can be easily represented within a classical planning problem. We thus define a new class of planning problems, POCL planning problems, in order to adequately represent these requirements.

A *POCL planning problem* $\mathcal{P}$ is given by a tuple $(V, A, P_{s_I, g})$, where V is a finite set of propositional variables, A is a finite set of actions, and $P_{s_I, g}$ is an initial partial plan given by the tuple $(PS_I, \prec_I, CL_I)$. The partial plan $P_{s_I, g}$ must contain the two artificial actions *init* and *goal* as plan steps, and these actions must have been defined as above according to the initial state s_I and goal description g . Other than this requirement that $\{init, goal\} \subseteq PS_I$, $P_{s_I, g}$ can represent any partial plan. For our purposes, it will represent the partial plan that is a search node generated by a POCL planner, for use as the initial partial plan of POCL heuristics.

We can now define POCL solutions to POCL planning problems. Such a solution must be a *refinement* of $P_{s_I, g}$, meaning that it may only add plan steps, orderings, and causal links to $P_{s_I, g}$, but not change any of its contents. The idea of POCL search procedures which use repeated refinements, creating new POCL planning problems at each step, was introduced by Kambhampati (1997), who named it *refinement planning*.

Definition 3. A partial POCL plan $P_R = (PS_R, \prec_R, CL_R)$ is called a *refinement* of another partial POCL plan $P_I = (PS_I, \prec_I, CL_I)$ if and only if $PS_I \subseteq PS_R$, $\prec_I \subseteq \prec_R$, and $CL_I \subseteq CL_R$.

Definition 4. A partial POCL plan P is called a *POCL plan* (also *POCL solution*) for

2 Background

a POCL planning problem $\mathcal{P} = (V, A, P_{s_I, g})$ if and only if it is a refinement of $P_{s_I, g}$, every precondition is supported by a causal link, and there are no causal threats.

For our POCL heuristic, we solve a delete-relaxed POCL plan, emulating similar heuristics for state-based planning by taking the number of plan steps of the delete relaxed solution plan as its heuristic value. However, this creates an issue with causal links, as we still want delete relaxed actions to respect existing causal links, but causal threats are defined using the delete effects of actions. This means that actions with relaxed delete effects will never trigger a causal threat, even when they would prior to delete relaxation, meaning that we are relaxing both delete effects and causal links when we only want to be relaxing delete effects. Hence, we first describe a way of compiling causal links into variables in a way which enables them to function the same way even after delete effects are relaxed, before then defining a delete relaxed POCL planning problem.

For a POCL planning problem $\mathcal{P} = (V, A, P_{s_I, g})$ where $P_{s_I, g} = (PS_I, \prec_I, CL_I)$, we can *compile* a causal link $cl = (ps_p, v, ps_c) \in CL_I$ by adding a variable v_{cl} to V . If ps_p is not the *init* step, we add v_{cl} to $add(init)$ and $del(ps_p)$. If ps_c is not the *goal* step, we add v_{cl} to $add(ps_c)$. This means that v_{cl} is false between the steps ps_p and ps_c and true everywhere else, and so whenever v_{cl} is false, cl is actively protecting the variable v . To transfer this protective power to the variable v_{cl} , for all $a \in A$ where $v \in del(a)$ we add v_{cl} to $pre(a)$, noting that this will also modify plan steps whose action is a . This ensures that a causal threat cannot be raised, and that v_{cl} now serves the same function as the causal link cl . Hence, we can then remove cl from CL_I , completing the compilation of cl into a variable. Figure 2.4 shows how Figure 2.2 would have its causal links compiled. Note how this compilation removed all possibilities of causal threats.

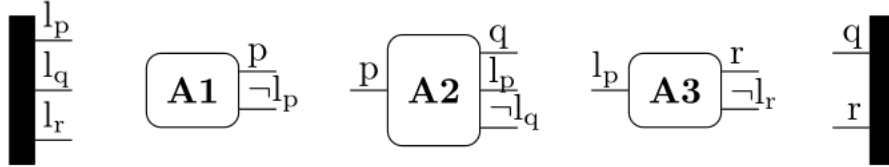


Figure 2.4: An example of a partial-order plan where causal link compilation has resolved threats.

A *delete relaxed* POCL planning problem $\mathcal{P}^+$ is the POCL planning problem $\mathcal{P} = (V, A, P_{s_I, g})$ where $P_{s_I, g} = (PS_I, \prec_I, CL_I)$, with all causal links in CL_I compiled, and $del(a) = \emptyset$ for all $a \in A$, meaning that all delete effects of actions are removed. Note that the plan steps already contained within the initial plan $P_{s_I, g}$ are unaffected, and so are their causal links due to compilation. Hence, only plan steps added in future refinements will have no delete effects, as they will draw from the now delete-relaxed set of actions in the POCL planning problem. This means that the implication of delete relaxation

of classical planning problems, that any action which becomes executable will forever remain executable, does not hold for the delete relaxation of POCL planning problems, as a plan step from $P_{s_I, g}$ could be ordered before such an executable action, deleting one of its preconditions and making it no longer executable. This added complexity is part of the reason why solving the delete relaxation of a POCL planning problem is NP-complete, whereas in classical planning it is polytime.

To solve planning problems, planners search an expanding tree of search nodes, using *heuristics* to pick which branches to prioritise exploration of. These heuristics usually output some estimate of solution length, often by solving a relaxed version of the problem from the given search node. As these heuristics need to be calculated for a growing set of search nodes, efficiency is critical, and so almost all planning heuristics run in polytime. In addition to efficiency, there are other metrics that are important to consider for heuristic effectiveness. *Admissible* heuristics never overestimate the number of actions required to meet the goal, guaranteeing optimality of the first found solution when expanding nodes in order of current actions used plus heuristic estimate. An admissible heuristic h_1 *dominates* another admissible heuristic h_2 if for any search node x , $h_1(x) \geq h_2(x)$. This means that it creates more accurate estimates, and so dominant heuristics are considered more powerful. An *informed* heuristic takes into consideration problem-specific information or current search progress in its calculations. We say that a heuristic is *more informed* than another if it uses more of the situational information available to it within its calculations.

2.2 Background on LPs and ILPs

Linear Programming (LP) is a mathematical optimisation technique for solving decision-making problems (Karloff, 1991). It converts problems into many-dimensional geometric shapes, where each *variable* involved in the problem represents one dimension of the LP, and each *constraint* of the problem is a hyperplane within that space, where one side is the region which *satisfies* the constraint, and the other side is the region which doesn't. The intersection of the satisfied regions for each constraint is called the *feasible region*, which is often a fully bounded shape, and represents every viable solution. The measure of optimality of a solution is represented in an LP by a directional vector, which points in the direction of better solutions. This is called the *objective function*, and by finding the point in the feasible region furthest in the direction of the objective function, we can find the optimal solution to any problem able to be represented as an LP. In practice, LPs are usually represented through matrices, but they can always be expanded into systems of linear equations which correspond to these visual descriptions.

Integer Linear Programming (ILP) is an extension of LP that allows for variables which aren't continuous, only holding integer values. This implicitly ensures that the feasible solutions can only be integers, picking only integer-valued points within the LP-feasible

2 Background

region. This allows ILP to encode a larger range of problems, as many problems have discrete components which require discrete variables. However, ILP is in a higher complexity class than LP, as integer variables make such problems much more difficult to solve and to optimise. Any ILP can be *relaxed* into a corresponding LP by removing the integer restrictions on the solution and all integer variables.

The problem of both finding a feasible solution and finding an optimal solution for LP was found to be achievable in polytime by [Khachiyan \(1979\)](#). However, for a polytime problem it is quite difficult to solve, and [Lawler and Labetoulle \(1978\)](#) alludes to discussion over whether solving an LP is a polytime or NP problem just a year before the first polytime algorithm was developed. In fact, the simplex algorithm, the most common algorithm before a polytime algorithm for solving LPs was found, is still the most common method for solving LPs. This is discussed by [Traub and Woźniakowski \(1982\)](#), who analyse the complexities in LP solving that allow for an NP algorithm to generally outperform a polytime one. This has meant that LP-based heuristics have the potential to be quite slow compared to other polytime heuristics. Finding feasible and optimal solutions for ILPs was shown to be NP-complete by [Karp \(1972\)](#). This result is more intuitive, as it is easy to see how a SAT problem, determining whether a logical boolean formula is satisfiable, can be reduced to an ILP with each boolean represented by a variable constrained to be 0 or 1.

Our heuristic for POCL planning, which accounts for all node information; delete effects, causal links, and orderings; has been proven to be an NP-complete problem by [Bercher \(2021\)](#), and so we can hence reduce this problem to an ILP in polytime. Our heuristic is concerned with finding a relaxed POCL plan for a POCL planning problem, which starts from an initial partial plan and delete-relaxes the domain of actions that can be added to it. Our heuristic is designed such that any initial partial plan can be converted to an ILP problem where the optimal solution is the minimum number of additional actions required to create a relaxed POCL solution from the initial partial plan. An ILP-based approach is particularly useful for propositional STRIPS planning, as propositional variables can be represented as 0–1 variables in an ILP, and time steps can be represented by integer values as well.

2.3 Formalisation of LPs and ILPs

Here we provide the formal framework for Integer Linear Programming which will be used for the remainder of this report. We consider the problem of finding the optimal solution to Integer Linear Programming models. We first formally define Linear Programming, and then extend these definitions to account for integer variables and solutions. We follow the canonical notation described by [Karloff \(1991\)](#) on page 5.

2.3 Formalisation of LPs and ILPs

A *linear program* (LP instance) is given by a tuple (x, c, A, b) , where $x \in \mathbb{R}^n$ is a vector of n variable values, $c \in \mathbb{R}^n$ is a vector of n coefficients for x , $A \in \mathbb{R}^k \times \mathbb{R}^n$ is a matrix representing k inequality constraints and the n coefficients of variables for each constraint, and $b \in \mathbb{R}^k$ is a vector of values on the right hand side of each constraint. A linear program is *feasible* if $Ax \geq b$ and $x \geq 0$, and *optimal* if it is feasible and there is no other feasible linear program (x', c, A, b) such that $c^T x' < c^T x$. The *solution* of a feasible linear program is the value $c^T x$.

A *linear programming problem* (LP problem) is the problem of finding a vector x for given tuple (c, A, b) which creates an optimal linear program (x, c, A, b) . This is done by minimizing an *objective function* $\min c^T x$ subject to the constraints $Ax \geq b, x \geq 0$. To make the contents of a linear programming problem more clear than a tuple, it can also be written in the following form:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax \geq b \\ & x \geq 0 \end{array}$$

Definition 5. A vector $x \in \mathbb{R}^n$ is a *feasible solution* to a linear programming problem (c, A, b) if and only if the linear program (x, c, A, b) is feasible.

Definition 6. A vector $x \in \mathbb{R}^n$ is an *optimal solution* to a linear programming problem (c, A, b) if and only if the linear program (x, c, A, b) is optimal.

Linear programming is limited in its applications due to the restriction of continuous variables. Since linear programming problems are solvable in polytime, they cannot be used for our delete relaxation heuristic for POCL planning, which is NP-complete, without an exponential encoding. However, since ILP is also NP-complete, and offers a convenient way to represent propositions and time steps as integers, it is relevant to our heuristic.

An *integer linear program* (ILP instance) is given by a tuple (x, c, A, b) , where $x \in \mathbb{Z}^n$ is a vector of n integer variable values, $c \in \mathbb{R}^n$ is a vector of n coefficients for x , $A \in \mathbb{R}^k \times \mathbb{R}^n$ is a matrix representing k inequality constraints and the n coefficients of variables for each constraint, and $b \in \mathbb{R}^k$ is a vector of values on the right hand side of each constraint. An integer linear program is *feasible* if $Ax \geq b$ and $x \geq 0$, and *optimal* if it is feasible and there is no other feasible integer linear program (x', c, A, b) such that $c^T x' < c^T x$. The *solution* of a feasible integer linear program is the value $c^T x$. This is all analogous to linear programs, with the exception of x , which now contains integer valued variables. Note that often within the literature, ILPs are used to refer to what are more formally *mixed integer linear programs* (MILPs), which combine the concepts of LPs and ILPs together to allow for both integer-valued and continuous variables. However, in this report we do not require such a combination, and so when we mention ILPs we specifically

2 Background

mean that all variables must be integer-valued.

An *integer linear programming problem* (ILP problem) is the problem of finding an integer-valued vector x for given tuple (c, A, b) which creates an optimal integer linear program (x, c, A, b) . It is defined and written in the same form as a linear programming problem, except with all mentions of a linear program replaced with an integer linear program.

Definition 7. A vector $x \in \mathbb{Z}^n$ is a *feasible solution* to an integer linear programming problem (c, A, b) if and only if the integer linear program (x, c, A, b) is feasible.

Definition 8. A vector $x \in \mathbb{Z}^n$ is an *optimal solution* to an integer linear programming problem (c, A, b) if and only if the integer linear program (x, c, A, b) is optimal.

While an ILP problem represents a single problem, often we want to use ILP to solve a class of problems. This requires a transformation which takes a set of problem parameters that define a specific instance of the problem class, and produces an ILP problem from them. This is called an *ILP model*, and is defined as a function of the form $ILP(p) = (c, A, b)$, where p is a parameter of any type, and (c, A, b) is an ILP problem. An example of a problem class for which an ILP model would be useful is sudokus, where the given digits in a specific sudoku puzzle would be the parameters for the ILP model used to generate an ILP problem which represents that sudoku puzzle. We use an ILP model to define our heuristics, as when solving a relaxed POCL planning problem, we can use the problem itself, $\mathcal{P}^+$, as the parameter for the ILP model, which is then translated into an ILP problem.

Another useful concept in ILP is *LP relaxation*. An LP relaxation of an ILP problem is a corresponding LP problem where variables are no longer restricted to being integer-valued, and become continuous. LP relaxations are very useful ways of finding simple relaxations to NP-hard problems, as any problem that can be represented as an ILP problem can very easily be relaxed into an LP problem.

Related Work

This report relates to a large body of prior work in planning, including: various planning approaches, POCL heuristics, ILP-based heuristics, and delete-relaxation. Much of the related work regarding the history of POCL planning and its comparative strengths and weaknesses to state-based planning has been covered in the introduction. Hence, this section will go more in-depth, focusing on specific prior works. We first focus on the development of ILP usage in planning within a variety of planning approaches, including their use for delete-relaxation heuristics. We then cover the development of POCL heuristics, before explaining complexity results regarding the combination of these two fields, and how this relates to our heuristic.

3.1 ILPs in Planning

An early use of ILPs in planning was proposed by [Bylander \(1997\)](#), who developed the Lplan heuristic, the first admissible heuristic for partial-order planning. This technique utilises the fact that classical planning is NP-complete for plans of polynomially bounded length, as shown in Corollary 1 of the paper by [Turner \(2002\)](#). To find the optimal plan length, an ILP is created which solves the planning problem with a bound of 1, and if it is infeasible a new ILP is created for a bound of 2, with this process repeated until the first feasible bound is found, which hence gives the minimum plan length. However, this formulation was found to be inefficient as a direct method for solving planning problems. Instead, the ILP is relaxed to an LP for use as a heuristic. As an LP allows non-integer variable values, this allows for strange solutions, such as half of two different actions being executed at one time step. It also still contains efficiency concerns due to having to run the LP multiple times for each possible bound on solution length. The main advantage of the Lplan heuristic is that it is more informed than any other heuristic it was compared to at the time, and while this provided runtime advantages over other partial-order planning methods in some instances, its higher runtime outweighed its in-

3 Related Work

formation advantage in most cases even when it was first published.

While future papers on the use of ILPs for planning shifted away from partial-order planning as it became less popular, a shift that was explained in the introduction. However, there were still many papers focused on modeling classical planning problems, and relaxations of those problems, as ILPs, as well as the use of these ILPs as heuristics. Many early attempts at using ILPs for planning aimed to solve planning problems directly with ILPs, using similar techniques to that of [Bylander \(1997\)](#), where successive ILPs were used to check each possible bound until a solution was found. The culmination of this type of ILP solver was the Optiplan model by [Van den Briel and Kambhampati \(2005\)](#). Optiplan combined the *state-change formulation* method of [Vossen et al. \(1999\)](#), which used variables to directly express the addition, deletion, and persistence of propositions over time, with additional refinement strategies developed by [Dimopoulos \(2001\)](#). However, SAT planning methods, which used a similar technique of iterative checking of higher bounds on plan size, but using satisfiability formulas instead of ILPs, continued to outperform ILP-based methods, and so research into ILP-based planners slowed.

Despite the lack of promise in the field of ILP planners, the use of LPs for creating admissible heuristics, as with the Lplan heuristic, still remained potentially promising. An admissible heuristic using an ILP model was developed by [Van Den Briel et al. \(2007\)](#), which completely relaxes all ordering constraints, including allowing the effects of one actions to be used as preconditions for an earlier action, to create an ILP formulation. This formulation is then relaxed to an LP and used as a heuristic. This heuristic was shown to outperform Lplan on many planning instances. The SEQ Heuristic, introduced by [Bonet \(2013\)](#) uses directed bipartite graph relaxations of planning problems to generate relaxed solutions which provide a minimal number of each action that must be executed to reach a goal state. By counting the total of all of these actions, an admissible heuristic is found. This calculation requires an ILP, but using its LP relaxation as a heuristic was found to be competitive with state-of-the-art optimal planning heuristics. A third LP-based heuristic was a method for representing pattern database heuristics developed by [Pommerening et al. \(2013\)](#). Pattern database heuristics store a large number of perfect cost estimates for small subproblems, and combine these subproblems together to estimate the cost of larger problems. An important component of pattern database heuristics is how to determine which patterns are relevant. This paper presents the Post-Hoc Optimisation heuristic, which links patterns that modify certain variables to actions which modify the same variables, joining them into operator groups and obtaining bounds on the minimum number of actions from each group required for a solution. This heuristic outperformed previous pattern database heuristic methods. You may note that all three of the heuristics discussed utilise some form of counting the number of each action required for some lower bound on solution length. This was also recognised by [Pommerening et al. \(2014\)](#), who developed a more general *operator counting* heuristic, which was a framework for developing LPs which could represent any

of the prior three heuristics, as well as other similar heuristics. It could also combine multiple heuristics that fit into the operator counting framework together, allowing it to dominate all individual operator counting heuristics, as synergies were created by running them together. This technique was found to be very powerful, and is utilised to relax some constraints of our ILP model.

Another use case, using ILPs to compute the optimal delete-relaxation of a state-based planning problem (h^+), was developed by [Imai and Fukunaga \(2015\)](#). [Bylander \(1994\)](#) has shown that computing h^+ precisely is an NP-complete problem, and so [Imai and Fukunaga \(2015\)](#) used an ILP to calculate it, which was found to be competitive with other techniques of computing h^+ . The LP relaxation of this ILP was shown to produce strong estimates of h^+ , and so is an effective heuristic for state-based planning. This heuristic was used as the basis of a hierarchical task network (HTN) planning heuristic by [Höller et al. \(2020\)](#), which computed a delete and ordering relaxed (DOR) HTN problem with an ILP as a heuristic. What is particularly relevant about this paper is that it also features a type of delete relaxation which, while polytime in state-based planning, is NP-complete in HTN planning, as is the case for POCL planning. The DOR heuristic was more informed than any other HTN heuristics in the literature, and remained competitive despite being NP-complete. The LP relaxation of the heuristic also performed well.

3.2 POCL Heuristics

The Relax heuristic created by [Nguyen and Kambhampati \(2001\)](#), and the Additive heuristic created by [Younes and Simmons \(2003\)](#), as covered previously in this report, have been two of the most dominant heuristics in POCL planning for over 20 years. However, because they ignore a large amount of partial-order plan information in order to run in polytime, they cannot take advantage of many of the benefits of POCL planning, as was argued in the introduction. This issue was noted by [Vidal and Geffner \(2006\)](#), who explained how previous POCL search techniques lack effective pruning techniques which can handle the unique branching scheme of POCL planning. They present a method of pruning which analyses how future actions could be added to the current partial plan, and prune any branches which don't fit with these constraints. Many of these constraints are similar to the constraints used in our heuristic, which prunes branches whenever the heuristic is infeasible, and so the effectiveness of their method provides promise for the effectiveness of our heuristic.

In a more direct relation to our heuristic, [Bercher et al. \(2013\)](#) developed the only previous heuristic which accounts for most POCL search node information. This was done by translating a partial plan into a classical planning problem, which could then run state-based relaxations and heuristics. This allowed for state-based heuristics to be used

3 Related Work

in POCL planning, including admissible ones, where the only prior admissible heuristic for partial order planning was the Lplan heuristic. These heuristics were slow compared to heuristics built specifically for POCL, with much of this being due to slow translation times, an issue that isn't relevant for our heuristic. However, it did show promise for potential reduction in the number of explored search nodes.

Complexity is an important consideration when developing heuristics for planning. The problem of satisfiability, determining whether a planning problem instance is solvable, and minimization, determining whether there is a solution using at most n actions, are both PSPACE-complete in Propositional STRIPS planning without severe restrictions (Bylander, 1994). Hence, to differentiate different planning systems, heuristic effectiveness and complexity is an important consideration. The theoretical background for our heuristic comes from the research by Bercher (2021), which in Table 1 lists the complexity of many different relaxation possibilities for POCL planning. It finds that by only relaxing the delete effects of insertable actions; and not relaxing any actions, causal links, and ordering constraints within a partial plan; an NP-complete heuristic can be developed. Hence, such a heuristic is computable with an ILP model. This method allows for all POCL search node information to be considered, but necessitates NP-complete heuristics.

Well-Informed Delete Relaxation Heuristics for POCL Planning

In this chapter we define the ILP model $ILP(\mathcal{P}^+)$, which is the ILP formulation of a delete relaxed POCL planning problem $\mathcal{P}^+$. The goal of this ILP model is to ensure that there exists a relaxed POCL plan P for $(\mathcal{P}^+)$ if and only if there is a corresponding feasible solution to $ILP(\mathcal{P}^+)$ whose cost is equivalent to the number of actions added to create P . This mapping will be injective, there may be more than one feasible solution corresponding to each relaxed POCL plan. We then define the ILP model $ILP_c(\mathcal{P}^+)$, which is further relaxed by utilising counting constraints. Note that $\mathcal{P}^+$ represents a search node where current delete effects and causal links are not relaxed, future additional actions are delete-relaxed, and causal links are compiled to allow them to be respected by additional actions. Due to this compilation, causal links are represented as propositions, and will not be treated differently to any other proposition in the model. These models are loosely based on the ILP model for a delete relaxed planning graph in classical planning by [Imai and Fukunaga \(2015\)](#).

4.1 ILP Formulation of a Delete Relaxed POCL Problem

In this section, we define the ILP model $ILP(\mathcal{P}^+)$, where $\mathcal{P}^+ = (V, A, P_{s_I, g})$ and $P_{s_I, g} = (PS_I, \prec_I, CL_I)$. We will do this by first defining the variables, each of which will represent a member of x in the optimal ILP instance, and then the objective function, which represents $c^T x$, and finally the constraints, which represent $Ax \geq b$. After the formulation, we will prove the validity of the ILP model.

The first step of transforming the input search node into an ILP problem is to remove

4 Well-Informed Delete Relaxation Heuristics for POCL Planning

the steps *init* and *goal* from PS_I , and also all orderings including either of those steps within $\prec_I$. This is because ILP variables relating to them will be added in separate sets, as they are handled differently. The model also often makes reference to s_I and g instead for simplicity. Note that since all causal links have already been compiled, the causal links connecting to these steps will still be respected.

Firstly, we define the variables of $ILP(\mathcal{P}^+)$. Each variable v has a domain of allowable integer values $dom(v)$. Some variable domains include an upper bound ub , which is an upper bound on the number of actions and steps in the solution plan. For the initial search node, which contains no plan steps other than *init* and *goal*, the length of the planning graph is used as an upper bound. Successive search nodes use an upper bound derived from the solution length of their parent. With a single refinement, the solution length of a search node on a valid branch can only increase by the number of new delete effects added in the refinement, and so this informs the bounds of child search nodes.

Proposition Usage Variables: For each proposition v , we have a variable for its initial instance, and for each possible future instance after each deletion of v . These variables will have a value of 1 if and only if the proposition instance is achieved.

$$PU = \{PU_{v,init} \mid v \in V\} \cup \{PU_{v,ps} \mid ps \in PS_I, v \in del(ps)\}$$

We also consider the subset of these variables which represent the initial state s_I .

$$PI = \{PU_{v,init} \mid v \in V, v \in s_I\}$$

These variables have two separate domains, as propositions in the initial state must be achieved.

$$\begin{aligned} \forall PU_{v,ps} \in PI : dom(PU_{v,ps}) &= \{1\} \\ \forall PU_{v,ps} \in PU \setminus PI : dom(PU_{v,ps}) &= \{0, 1\} \end{aligned}$$

Action Usage Variables: For each action a , we have one variable for its initial execution, and additional variables for each possible execution required to achieve its effect propositions after they are deleted. These variables will have a value of 1 if and only if the action instance is executed.

$$\begin{aligned} AU &= \{AU_{a,init,init} \mid a \in A\} \\ &\cup \{AU_{a,v,ps} \mid ps \in PS_I, a \in A, v \in del(ps) \cap add(a)\} \\ \forall AU_{a,v,ps} \in AU : dom(AU_{a,v,ps}) &= \{0, 1\} \end{aligned}$$

4.1 ILP Formulation of a Delete Relaxed POCL Problem

Action Effect Variables: For each action usage variable $UA_{a,v,ps}$, we have a variable for each add effect of its associated action a . These variables will have a value of 1 if and only if the given action instance is executed and is a first achiever of the given effect.

$$AE = \{AE_{a_f,v_f,ps_f,v_a} \mid AU_{a_f,v_f,ps_f} \in AU, v_a \in add(a_f)\}$$

$$\forall AE_{a_f,v_f,ps_f,v_a} \in AE : dom(AE_{a_f,v_f,ps_f,v_a}) = \{0, 1\}$$

Step Effect Variables: For each plan step ps , we have a variable for each of its add effects. These variables will have a value of 1 if and only if the given plan step is a first achiever of the given effect.

$$SE = \{SE_{ps,v} \mid ps \in PS_I, v \in add(ps)\}$$

$$\forall SE_{ps,v} \in SE : dom(SE_{ps,v}) = \{0, 1\}$$

Proposition Time Variables: For each proposition usage variable $PU_{v,ps}$, we have a time variable. These variables will have a value of the execution time of $PU_{v,ps}$ if $PU_{v,ps} = 1$, and are otherwise unconstrained.

$$PT = \{PT_{v,ps} \mid PU_{v,ps} \in PU\}$$

$$\forall PT_{v,ps} \in PT : dom(PT_{v,ps}) = \{0, \dots, ub\}$$

Action Time Variables: For each action usage variable $AU_{a,v,ps}$, we have a time variable. These variables will have a value of the execution time of $AU_{a,v,ps}$ if $AU_{a,v,ps} = 1$ (i.e. if this action instance is used in the plan), and are otherwise unconstrained.

$$AT = \{AT_{a,v,ps} \mid AU_{a,v,ps} \in AU\}$$

$$\forall AT_{a,v,ps} \in AT : dom(AT_{a,v,ps}) = \{0, \dots, ub\}$$

Step Time Variables: For each plan step ps , we have a time variable. These variables will have the value of the execution time of ps .

$$ST = \{ST_{ps} \mid ps \in PS_I\}$$

$$\forall ST_{ps} \in ST : dom(ST_{ps}) = \{0, \dots, ub\}$$

We also define a set of step deletion constants which connect propositions to the steps which delete them. Note that these aren't ILP variables as their values can be determined directly from the input search node. They are used as constants within constraints.

$$SD = \{SD_{ps,v} \mid ps \in PS_I, v \in del(ps)\}$$

$$\forall SD_{ps,v} \in SD : dom(SD_{ps,v}) = \{0, 1\}$$

The objective function returns the minimum number of added actions required to complete the partial plan, and is defined as:

$$\min \sum_{AU_{a,v,ps} \in AU} AU_{a,v,ps}$$

Finally, we define the constraints of $ILP(\mathcal{P}^+)$. The value M used in some of these constraints as a large value used to render constraints either necessarily true or necessarily false under some condition, and is set to $M = ub + 2$. Note that a difference of one will cause issues with constraints that involve a difference of 1 when connecting actions with effect propositions on the following time step. The operators $\vee$ and $\wedge$ will be used as a shorthand within constraints, as it is well known how these operators can be encoded in an ILP.

Goals Achieved (C1): For each goal proposition, ensure that there is at least one instance achieved after all of its deletions.

$$\forall v_g \in g : \bigvee_{PU_{v_g,ps_a} \in PU} \left((PU_{v_g,ps_a} = 1) \wedge \left(\bigwedge_{SD_{ps_b,v_g} \in SD} (PT_{v_g,ps_a} - ST_{ps_b} \geq 1) \right) \right)$$

First Achievers Must be Executed (C2): For each action effect instance, ensure that if the action instance is the first achiever of this effect, the action instance is used.

$$\forall AE_{a_i,v_i,ps_i,v_i} \in AE : AU_{a_i,v_i,ps_i} - AE_{a_i,v_i,ps_i,v_i} \geq 0$$

Propositions are Preceded by an Achiever (C3): For each proposition instance that is not in SI , ensure that the time step prior to the instance being achieved contains an achiever for the proposition.

$$\begin{aligned}
 & \forall PU_{v_i, ps_i} \in PU \setminus SI : \\
 & (PU_{v_i, ps_i} = 0) \vee \\
 & \bigvee_{AE_{a_f, v_f, ps_f, v_i} \in AE} ((AE_{a_f, v_f, ps_f, v_i} = 1) \wedge (PT_{v_i, ps_i} - AT_{a_f, v_f, ps_f} = 1)) \vee \\
 & \bigvee_{SE_{ps, v_i} \in SE} ((SE_{ps, v_i} = 1) \wedge (PT_{v_i, ps_i} - ST_{ps} = 1))
 \end{aligned}$$

Actions are Preceded by their Preconditions (C4): For each action instance, and each of its preconditions, ensure that there is at least one instance of the precondition achieved before the action instance, and no deletions of the precondition in-between the times of the precondition instance and the action instance.

$$\begin{aligned}
 & \forall AU_{a_i, v_i, ps_i} \in AU, \forall v_p \in pre(a_i) : \\
 & (AU_{a_i, v_i, ps_i} = 0) \vee \bigvee_{PU_{v_p, ps_a} \in PU} \left((PU_{v_p, ps_a} = 1) \wedge (AT_{a_i, v_i, ps_i} - PT_{v_p, ps_a} \geq 0) \wedge \right. \\
 & \left. \left(\bigwedge_{SD_{ps_b, v_p} \in SD} ((PT_{v_p, ps_a} - ST_{ps_b} \geq 1) \vee (ST_{ps_b} - AT_{a_i, v_i, ps_i} \geq 1)) \right) \right)
 \end{aligned}$$

Steps are Preceded by their Preconditions (C5): For each plan step instance, and each of its preconditions, ensure that there is at least one instance of the precondition achieved before the plan step, and no deletions of the precondition in-between the times of the precondition instance and the plan step.

$$\begin{aligned}
 & \forall ps_i \in PS, \forall v_p \in pre(ps_i) : \\
 & \bigvee_{PU_{v_p, ps_a} \in PU} \left((PU_{v_p, ps_a} = 1) \wedge (ST_{ps_i} - PT_{v_p, ps_a} \geq 0) \wedge \right. \\
 & \left. \left(\bigwedge_{SD_{ps_b, v_p} \in SD} ((PT_{v_p, ps_a} - ST_{ps_b} \geq 1) \vee (ST_{ps_b} - ST_{ps_i} \geq 1)) \right) \right)
 \end{aligned}$$

4 Well-Informed Delete Relaxation Heuristics for POCL Planning

Step Interference (C6): For each pair of steps (ps_i, ps_j) where one deletes a precondition of the other, ensure they cannot execute in parallel. This requires a set of interfering step pairs to be defined.

$$SI = \{SI_{ps_i, ps_j} \mid ps_i \in PS_I, ps_j \in PS_I, del(ps_i) \cap pre(ps_j) \neq \emptyset \vee pre(ps_i) \cap del(ps_j) \neq \emptyset\}$$

We can define the corresponding constraints:

$$\forall SI_{ps_i, ps_j} \in SI : ST_{ps_i} \neq ST_{ps_j}$$

Step Inconsistent Effects (C7): For each pair of steps (ps_i, ps_j) where an add effect of one step is a delete effect of the other, ensure they cannot execute in parallel. This requires a set of inconsistent step pairs to be defined.

$$SIE = \{SIE_{ps_i, ps_j} \mid ps_i \in PS_I, ps_j \in PS_I, del(ps_i) \cap add(ps_j) \neq \emptyset \vee add(ps_i) \cap del(ps_j) \neq \emptyset\}$$

We can then define the corresponding constraints:

$$\forall SIE_{ps_i, ps_j} \in SIE : ST_{ps_i} \neq ST_{ps_j}$$

Step Orderings (C8): For each step ordering in $\prec_I$, we enforce that this ordering holds.

$$\forall (ps_i \prec ps_j) \in \prec_I : ST_{ps_j} - ST_{ps_i} \geq 1$$

We now prove that for any solution to a delete-relaxed POCL planning problem $\mathcal{P}^+$, the corresponding instance of $ILP(\mathcal{P}^+)$ will be feasible solution. We then prove the inverse.

Theorem 1. *Given a delete-relaxed POCL planning problem $\mathcal{P}^+$, a POCL solution P for $\mathcal{P}^+$, and the action sequence $\bar{a}$ of any linearization of P , there is a corresponding feasible solution to $ILP(\mathcal{P}^+)$.*

Proof. For $P = (PS, \prec, CL)$ to be a POCL solution all of its linearizations must be total-order solutions. Hence, $\bar{a}$ must result in a goal state, of which g must be a subset. For this to be true, each $v_g \in g$ must be at some point achieved after all of its deletions. This means that one PU variable corresponding to v_g must be achieved after all steps which delete v_g are executed, and so its corresponding PT variable must be after all deleting ST variables. Thus, constraint C1 must be satisfied.

4.1 ILP Formulation of a Delete Relaxed POCL Problem

Constraint C2 is satisfied as for any action in $\bar{a}$ to achieve its effect and hence have its AE variable set to 1, it also has to be executed and so have its AU variable set to 1.

Each achiever will set an instance of the achieved proposition to true at the next time step, and only initial propositions don't have a corresponding achiever. So AE/SE variables which are set to 1 are paired with PU variables set to 1, where $AT/ST + 1 = PT$. Thus, constraint C3 holds.

Because P is a POCL solution, every precondition must be supported by a causal link. This causal link prevents threats by requiring that all steps which could potentially delete that precondition to either be ordered before the producer step or after the consumer step. Translating this to the ILP model, each precondition requires at least one instance where all deleting steps are either ordered before the instance is achieved or after the action/step the precondition is used for. This matches with constraints C4 and C5, ensuring that they are satisfied.

Constraints C6 and C7 are trivial for a total-order action sequence, as all steps run at a unique time, and so any time inequality constraint between steps will be satisfied.

Finally, constraint C8 also holds, as step orderings in $\prec$ are required to hold for any linearization of P , and so they will also hold in the corresponding ILP problem.

Theorem 2. *Given a feasible solution for $ILP(\mathcal{P}^+)$, the plan step sequence $\bar{ps}$ obtained by ordering all action instances $\{a \mid AU_{a,v,ps} = 1\}$ (converted to plan steps with labels (a, v, ps)) and steps $ps \in PS$ by ascending order of their respective time variables AT and ST (with ties broken randomly), is a valid linearization of a POCL solution P for $\mathcal{P}^+$.*

Proof. We must show that such a plan $P = (V, A, P_{s_I, g})$ satisfies the requirements that every precondition is supported by a causal link, and all causal threats are resolved. To do this, we first construct $P = (PS, \prec, CL)$. Firstly, we set PS to the set of all plan steps in $\bar{ps}$, as well as the *init* and *goal* steps, constructed from s_I and g . In addition to the necessary *init* and *goal* orderings, we add orderings to $\prec$ for all orderings within $\bar{ps}$. I.e. for each ps_i in the sequence $\bar{ps}$, all steps ps_j where $j < i$ add an ordering (ps_j, ps_i) to $\prec$, and all steps ps_j where $i > j$ add an ordering (ps_i, ps_j) to $\prec$. Finally, for each precondition $ps_c \in PS, v \in pre(ps_c)$, a causal link ps_p, v, ps_c is added to CL , where ps_c corresponds to the plan step prior to and closest to ps_c in $\bar{ps}$ which has an action/step effect variable for v in the ILP solution equal to 1. I.e. ps_c is the closest achiever of v prior to ps_p . If no such achiever exists then $ps_p = init$. Because $\bar{ps}$ contains all plan steps from P and follows all orderings in $\prec$, it is a valid linearization of P .

Now that P is constructed, we first show that all preconditions are supported by a causal link. Since it has been shown how all required causal links are constructed from a given ps_c and v , we only need to prove that the producer steps are valid, i.e. $\forall(ps_p, v, ps_c) \in CL : v \in add(ps_p)$. Since ps_p corresponds to a AE or SE variable for v , and these variables only exist when v is an add effect for the action/step of the variable, when such a variable exists the causal link will always be valid. When there is no such variable, $ps_p = init$, and so we must prove that $v \in s_I$. From constraint C3, we know that if a proposition instance has no corresponding AE or SE variables set to 1, the constraint can only be satisfied if the proposition instance is never achieved, or the proposition instance is from the initial state (in which case no C3 constraint is created, and so C3 trivially holds). We know from constraints C4 and C5 (or C1 if $ps_c = goal$) that since $v \in pre(ps_c)$, and ps_c is executed and hence its AU variable is set to 1 if it has one, that at least one instance of v must be achieved (to meet the $(PU_{v_p, ps_a} = 1)$ part of the constraint). Hence, that instance must be in the initial state, and so $v \in s_I$ holds. Thus, all preconditions in P are supported by a causal link.

The second and final requirement is that all causal threats in P are resolved. This requires that for all causal links, potentially threatening steps are ordered before the producer step or after the consumer step, i.e. $\forall(ps_p, v, ps_c) \in CL, ps_t \in PS : v \in del(ps_t) \implies ((ps_t, ps_p) \in \prec \vee (ps_c, ps_t) \in \prec)$. Assume for a contradiction that such a ps_t exists which is ordered in-between ps_p and ps_c . Constraint C6 ensures that ps_t can't be ordered directly before ps_c in $\overline{ps}$ by running in parallel with ps_c , as they would necessarily interfere. Constraint C7 ensures that ps_t can't be ordered directly after ps_p in $\overline{ps}$ by running in parallel with ps_p , as they would necessarily have inconsistent effects. Hence, ps_t must already be ordered in-between ps_p and ps_c before ties are broken. From constraints C4 and C5, since ps_t and ps_c cannot run in parallel, there must be at least one instance of v which is ordered after ps_t and before ps_c . However, ps_p has been defined as the last achiever of v before ps_c , and so for constraint C3 to hold, any instance of v which is timed before ps_c must also be before ps_t , creating a contradiction. Therefore, $\forall(ps_p, v, ps_c) \in CL, ps_t \in PS : v \in del(ps_t) \implies ((ps_t, ps_p) \in \prec \vee (ps_c, ps_t) \in \prec)$, and so all causal threats in P are resolved.

Corollary 1. *Given an optimal solution for $ILP(\mathcal{P}^+)$, the plan step sequence $\overline{ps}$ obtained by ordering all action instances $\{a \mid AU_{a,v,ps} = 1\}$ (converted to plan steps with labels (a, v, ps)) and steps $ps \in PS$ by ascending order of their respective time variables AT and ST (with ties broken randomly), is a valid linearization of a POCL solution P for $\mathcal{P}^+$ with a minimum number of plan steps.*

4.2 ILP Formulation of a Counting-Based Relaxation

The ILP model $ILP(\mathcal{P}^+)$ is not effective for solving planning problems, as its restrictive constraints make it incapable of solving any reasonably sized planning problem. This

4.2 ILP Formulation of a Counting-Based Relaxation

will be further discussed in the evaluation section, with possible solutions in the future work section. Hence, we use counting-based constraints, as popularised in ILP heuristics by Pommerening et al. (2014), to define a further relaxed heuristic $ILP_c(\mathcal{P}^+)$ which is reasonable to use in evaluations. This heuristic uses the same variables and objective function, so below we only list the new constraints. Later, we discuss in what way this relaxes the problem further than regular delete relaxation.

$ILP_c(\mathcal{P}^+)$ does not use $\vee$ and $\wedge$ operators as it instead focuses on summations and counting-based constraints, but it does require additional auxiliary variables. These variables will be described when necessary to represent the truth value of a linear inequality. For example a variable $v = (y - z \geq 0)$ would have a value of 1 when $y - z \geq 0$ and 0 otherwise. Fully notating these linear equality boolean variables requires multiple constraints to be added, but since the procedure to add these types of variables is well known, we will use the $v = (a^T x \geq b)$ shorthand for simplicity. The value M is also used in some of these constraints as a large value used to render constraints either necessarily true or necessarily false under some condition. It is set to $M = ub + 2$, noting that only adding one to the bound can cause issues with constraints that use a +1 to connect actions with effect propositions on the following time step.

Goals Achieved (C1): For each goal proposition, ensure that more instances of it are used than are deleted.

$$\forall v_g \in g : \sum_{PU_{v_g, ps} \in PU} PU_{v_g, ps} - \sum_{SD_{ps, v_g} \in SD} SD_{ps, v_g} \geq 1$$

First Achievers Must be Executed (C2): For each action effect instance, ensure that if the action instance is the first achiever of this effect, the action instance is used.

$$\forall AE_{a_i, v_i, ps_i, v_i} \in AE : AU_{a_i, v_i, ps_i} - AE_{a_i, v_i, ps_i, v_i} \geq 0$$

Propositions are Preceded by an Achiever (C3): For each proposition instance that is not in s_I , ensure that prior to the instance being achieved, the number of actions/steps achieving the proposition exceed the number of other instances of the proposition which are achieved. This requires defining three sets of additional linear equality boolean variables.

Firstly, we define a set of action effect achieved variables representing whether a given action effect achieves the proposition prior to the proposition time of the instance.

$$AEA = \{AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a} \mid PU_{v_i, ps_i} \in PU, AE_{a_f, v_f, ps_f, v_a} \in AE\}$$

4 Well-Informed Delete Relaxation Heuristics for POCL Planning

$$\begin{aligned} \forall AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a} \in AEA : dom(AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a}) &= \{0, 1\} \\ \forall AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a} \in AEA : AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a} &= (PT_{v_i, ps_i} - AT_{a_f, v_f, ps_f} \geq 0) \end{aligned}$$

Next, we define a set of step effect achieved variables representing whether a given step effect achieves the proposition prior to the proposition time of the instance.

$$\begin{aligned} SEA &= \{SEA_{v_i, ps_i, ps_j, v_j} \mid PU_{v_i, ps_i} \in PU, SE_{ps_j, v_j} \in SE\} \\ \forall SEA_{v_i, ps_i, ps_j, v_j} \in SEA : dom(SEA_{v_i, ps_i, ps_j, v_j}) &= \{0, 1\} \\ \forall SEA_{v_i, ps_i, ps_j, v_j} \in SEA : SEA_{v_i, ps_i, ps_j, v_j} &= (PT_{v_i, ps_i} - ST_{ps_j} \geq 0) \end{aligned}$$

Finally, we define a set of proposition instance achieved variables representing whether a given other instance of the proposition is achieved before the proposition time of the instance.

$$\begin{aligned} PIA &= \{PIA_{v, ps_i, ps_j} \mid PU_{v, ps_i} \in PU, PU_{v, ps_j} \in PU, ps_i \neq ps_j\} \\ \forall PIA_{v, ps_i, ps_j} \in PIA : dom(PIA_{v, ps_i, ps_j}) &= \{0, 1\} \\ \forall PIA_{v, ps_i, ps_j} \in PIA : PIA_{v, ps_i, ps_j} &= (PT_{v, ps_i} - PT_{v, ps_j} \geq 0) \end{aligned}$$

By utilising these variables we can now define the constraints:

$$\begin{aligned} \forall PU_{v_i, ps_i} \in PU : & \sum_{AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a} \in AEA} (AEA_{a_f, v_f, ps_f, v_a} AEA_{v_i, ps_i, a_f, v_f, ps_f, v_a}) \\ & + \sum_{SEA_{v_i, ps_i, ps_j, v_j} \in SEA} (SE_{ps_j, v_j} SEA_{v_i, ps_i, ps_j, v_j}) \\ & - \sum_{PIA_{v_i, ps_i, ps_j} \in PIA} (PU_{v_i, ps_j} PIA_{v_i, ps_i, ps_j}) + M(1 - PU_{v_i, ps_i}) \geq 1 \end{aligned}$$

Actions are Preceded by their Preconditions (C4): For each action instance, and each of its preconditions, ensure that prior to the action being executed the number of instances of the precondition achieved exceed the number of steps which delete the precondition executed. This requires defining two sets of additional linear equality boolean variables.

4.2 ILP Formulation of a Counting-Based Relaxation

Firstly, we define a set of action precondition satisfied variables representing whether a precondition instance is achieved prior to the action:

$$\begin{aligned} APS &= \{APS_{a_i, v_i, ps_i, v_j, ps_j} \mid AU_{a_i, v_i, ps_i} \in AU, v_j \in pre(a_i), PU_{v_j, ps_j} \in PU\} \\ \forall APS_{a_i, v_i, ps_i, v_j, ps_j} \in APS &: dom(APS_{a_i, v_i, ps_i, v_j, ps_j}) = \{0, 1\} \\ \forall APS_{a_i, v_i, ps_i, v_j, ps_j} \in APS &: APS_{a_i, v_i, ps_i, v_j, ps_j} = (PT_{v_j, ps_j} - AT_{a_i, v_i, ps_i} \geq 0) \end{aligned}$$

We also define a set of action precondition deleted variables representing whether the precondition is deleted by a given step prior to the action:

$$\begin{aligned} APD &= \{APD_{a_i, v_i, ps_i, v_j, ps_j} \mid AU_{a_i, v_i, ps_i} \in AU, ps_j \in PS_I, v_j \in del(ps_j)\} \\ \forall APD_{a_i, v_i, ps_i, v_j, ps_j} \in APD &: dom(APD_{a_i, v_i, ps_i, v_j, ps_j}) = \{0, 1\} \\ \forall APD_{a_i, v_i, ps_i, v_j, ps_j} \in APD &: APD_{a_i, v_i, ps_i, v_j, ps_j} = (AT_{a_i, v_i, ps_i} - ST_{ps_j} \geq 1) \end{aligned}$$

By utilising these variables we can now define the constraints:

$$\begin{aligned} \forall AU_{a_i, v_i, ps_i} \in AU, \forall v_p \in pre(a_i) &: \sum_{PU_{v_p, ps_j} \in PU} (PU_{v_p, ps_j} APS_{a_i, v_i, ps_i, v_p, ps_j}) \\ &- \sum_{APD_{a_i, v_i, ps_i, v_p, ps_j} \in APD} APD_{a_i, v_i, ps_i, v_p, ps_j} \\ &+ M(1 - AU_{a_i, v_i, ps_i}) \geq 1 \end{aligned}$$

Steps are Preceded by their Preconditions (C5): For each plan step, and each of its preconditions, ensure that prior to the plan step being executed the number of instances of the precondition achieved exceed the number of other steps which delete the precondition executed. This requires defining two sets of additional linear equality boolean variables.

Firstly, we define a set of step precondition satisfied variables representing whether a precondition instance is achieved prior to the step:

$$\begin{aligned} SPS &= \{SPS_{ps_i, v_j, ps_j} \mid ps_i \in PS_I, v_j \in pre(ps_i), PU_{v_j, ps_j} \in PU\} \\ \forall SPS_{ps_i, v_j, ps_j} \in SPS &: dom(SPS_{ps_i, v_j, ps_j}) = \{0, 1\} \end{aligned}$$

4 Well-Informed Delete Relaxation Heuristics for POCL Planning

$$\forall SPS_{ps_i, v_j, ps_j} \in SPS : SPS_{ps_i, v_j, ps_j} = (PT_{v_j, ps_j} - ST_{ps_i} \geq 0)$$

We also define a set of step precondition deleted variables representing whether the precondition is deleted by a given other step prior to the step:

$$SPD = \{SPD_{ps_i, v_j, ps_j} \mid ps_i \in PS_I, ps_j \in PS_I, v_j \in del(ps_j)\}$$

$$\forall SPD_{ps_i, v_j, ps_j} \in SPD : dom(SPD_{ps_i, v_j, ps_j}) = \{0, 1\}$$

$$\forall SPD_{ps_i, v_j, ps_j} \in SPD : SPD_{ps_i, v_j, ps_j} = (ST_{ps_i} - ST_{ps_j} \geq 1)$$

By utilising these variables we can now define the constraints:

$$\begin{aligned} \forall ps_i \in PS_I, \forall v_p \in pre(ps_i) : & \sum_{PU_{v_p, ps_j} \in PU} (PU_{v_p, ps_j} SPS_{ps_i, v_p, ps_j}) \\ & - \sum_{SPD_{ps_i, v_p, ps_j} \in SPD} SPD_{ps_i, v_p, ps_j} \geq 1 \end{aligned}$$

Step Interference (C6): For each pair of steps (ps_i, ps_j) where one deletes a precondition of the other, ensure they cannot execute in parallel. This requires a set of interfering step pairs to be defined.

$$SI = \{SI_{ps_i, ps_j} \mid ps_i \in PS_I, ps_j \in PS_I, del(ps_i) \cap pre(ps_j) \neq \emptyset \vee pre(ps_i) \cap del(ps_j) \neq \emptyset\}$$

We can define the corresponding constraints:

$$\forall SI_{ps_i, ps_j} \in SI : ST_{ps_i} \neq ST_{ps_j}$$

Step Orderings (C7): For each step ordering in $\prec_I$, we enforce that this ordering holds.

$$\forall (ps_i \prec ps_j) \in \prec_I : ST_{ps_j} - ST_{ps_i} \geq 1$$

These groupings of constraints correlate to the groupings of constraints in $ILP(\mathcal{P}^+)$, but are implemented when possible by counting the number of effects and propositions

4.2 ILP Formulation of a Counting-Based Relaxation

achieved and deleted in a given range of time. The notable exception is the step interference constraint, which isn't required for this model as both the deletion and addition of the same proposition at a parallel time step will be counted correctly by the other constraints.

The additional relaxation this model involves is that the ordering between achievement and deletion of variables is ignored. For example, if a proposition is deleted by three steps, but then achieved afterwards, that proposition is true and usable. However, this model would require 2 additional achieving actions, which could for example execute in parallel to the first achieving action. Similarly, if a proposition is achieved twice, but then deleted afterwards, that proposition is false. However, this model would treat it as usable for preconditions, because it has been achieved more times than it has been deleted. As the evaluation section will show, this relaxation is still significantly more informed than other POCL heuristics, and the additional relaxation compared to $ILP(\mathcal{P}^+)$ has a minimal impact in many domains. It is important to note that all input search node information is still utilised in this relaxation, including step deletions, causal links, and orderings.

Evaluation

This chapter explains the software and heuristics we are evaluating, and the benchmarks and hardware we tested them on. We then present the raw data obtained from these evaluations, along with plots for visualisation, and discussion of inferences we can make from these results.

5.1 Evaluated Software

We integrated our heuristics into the VHPOP POCL planning system by [Younes and Simmons \(2003\)](#). The $ILP(\mathcal{P}^+)$ heuristic could not solve any non-trivial problems from our test problems, and so we mention results for the $ILP_c(\mathcal{P}^+)$ heuristic, which we will refer to as the ILP_c heuristic. We compare the performance of planning using this heuristic against the state of the art heuristic within VHPOP, the ADDR heuristic, which is a version of the additive heuristic by [Younes and Simmons \(2003\)](#) which accounts for multiple instances of the same action. We also compare results for the state-based planner Fast Downward by [Helmert \(2006\)](#).

For ILP_c , we use the LCFR-Loc flaw selection strategy included natively within VHPOP ([Younes and Simmons, 2003](#)). LCFR-Loc solves causal threats before open conditions, which are preconditions not supported by a causal link. Among open conditions, LCFR-Loc favours local open conditions, which are open conditions for the same step as the most recently supported open condition. To determine the ordering of causal threats, open conditions which apply to the same step, or new open conditions when there are no local open conditions left, LCFR-Loc uses a *Least Cost Flaw Repair* strategy. Least Cost Flaw Repair prioritises flaws with smallest set of possible refinements to solve them, thus minimising the branching factor of the search space. Remaining ties are broken by LIFO, preferring newly discovered flaws, and then by chance if LIFO is also tied (for exam-

ple when two open preconditions are added at the same time by the same new plan step).

We chose the additive heuristic as a comparison as it is currently the best performing POCL heuristic, and is the only state-of-the-art POCL heuristic which is implemented natively within VHPOP. VHPOP implements multiple variations of the additive heuristic, and we chose ADDR as it is the most informed and effective version. For state-based heuristics to compare to, we use the add heuristic by [Geffner and Haslum \(2000\)](#), which the additive heuristic for POCL planning is an adaptation of, and the FF heuristic by [Hoffmann and Nebel \(2001\)](#), as it is a delete-relaxation based heuristic.

5.2 Benchmarks and Setup

We evaluate all heuristics on the first 10 instances of each STRIPS domain from the 3rd International Planning Competition (IPC3). IPC3 was chosen as it is the planning competition that VHPOP was originally evaluated on, and so VHPOP is built to handle the version of PDDL input files used in this competition. To evaluate the effectiveness of our heuristic for detecting unsolvable problems, we also tested the first 10 instances of STRIPS domains from the 1st Unsolvability International Planning Competition (UIPC1), held in 2016. This is the only well recognised international planning competition for unsolvable problems, as it has not yet been held again, and so is the best option for evaluating our heuristics on unsolvable domains. Our heuristic is built for grounded propositional planning, where all variables are booleans, but the domains in these evaluation sets are not initially grounded. Hence, all problems in our evaluation set are grounded using CPDDL ([Fišer, 2020](#)), which also infers some mutex propositions and prunes the grounded representation accordingly. For some domains in UIPC1, this grounding process leads to the domains becoming too large to be realistically solvable. We hence evaluate against the 6 domains from UIPC1 with the smallest grounded domain file sizes. Fast Downward accepts these grounded PDDL files as input, but translates them to SAS+ format before solving them. This translation time is included in the evaluation times for Fast Downward heuristics.

For each heuristic on each instance, we gave the planning system 10 minutes of CPU time on a 12 core Intel i7-12700H processor (2.30 GHz) with 8GB of RAM. The ILP_c heuristic is solved using version 22.1.1 of the CPLEX solver, restricted to 1 CPU core. The evaluation was run on a laptop within Windows Subsystem for Linux using the Ubuntu 22 distribution.

5.3 Empirical Results

Our results are divided into runtime, nodes generated (i.e. the size of the search space), and coverage (i.e. the number of problems solved within the time limit) on both solvable and unsolvable domains. Since our report is more focused on the potential benefits of more informed POCL heuristics, and our implementation hasn't been fully optimised for runtime, our focus is on the number of nodes generated by our heuristic. We are also interested in determining whether the extra information our heuristic utilises is effective for determining that problems are unsolvable, where other planners might struggle with exponentially growing state trees, and so we will also focus on the coverage results for unsolvable instances.

The coverage of solvable domains is shown in Table 5.1. As expected, the state-based heuristics have the highest coverage, and the ADDR heuristic has higher coverage than the ILP_c heuristic. There were no instances which a state-based heuristic could not solve but ADDR could, and there were no instances ADDR could not solve that ILP_c could. Table 5.2 shows the coverage of unsolvable domains. It is expected that the state-based heuristics have higher coverage than the ADDR heuristic, but we had hoped to find that our ILP_c heuristic has coverage over instances that the other heuristics did not. However, our heuristic performed very poorly on unsolvable instances, even compared to its relative weakness on solvable instances. All instances which ILP_c determined to be unsolvable were also determined to be unsolvable by all other heuristics in significantly less time. There are likely multiple factors which led to this result. Firstly, the ILP takes significantly longer per search node on unsolvable search nodes compared to solvable search nodes, as the ILP solver needs to perform a significantly larger amount of work to determine that there are no feasible solutions to an ILP. As unsolvable instances typically require the evaluation of more unsolvable search nodes, this disadvantage could outweigh the advantages of being more informed. Additionally, it could be that the additional relaxations in ILP_c over our standard delete-relaxed heuristic are detrimental to the benefits on unsolvable problems, while the restrictive constraints of our standard delete-relaxed heuristic still make it infeasible for use on any domains due to the evaluation time per search node. A more efficient implementation of the standard delete-relaxation heuristic could potentially still outperform other heuristics on some unsolvable domains, and our results do not conclusively prove that our hypothesis on the benefits of NP-complete heuristics for determining unsolvability is incorrect. More research is still required.

Since ILP_c could only solve two unsolvable instances, analysis of runtime and generated nodes for unsolvable instances is not relevant, as two instances is too few for any useful conclusions to be drawn apart from the low coverage. Hence, we now focus solely on the IPC 3 domains. Figure 5.1 shows a log scaled runtime comparison of ILP_c and ADDR on instances in IPC3 which they both solved before timeout. ADDR has a lower runtime on all instances, which is unsurprising, but the magnitude of this difference is significant.

5 Evaluation

Table 5.1: Coverage table for the first 10 instances of IPC3 domains for VHPOP and Fast Downward heuristics.

Domain	n	ILP _c	ADDR	FD-Add	FD-FF
Depots	10	1	4	8	9
DriverLog	10	4	10	10	10
FreeCell	10	0	1	10	10
Rovers	10	4	7	10	10
Satellite	10	3	10	10	10
ZenoTravel	10	2	10	10	10
Total	60	14	42	58	59

Table 5.2: Coverage table for the first 10 instances of 6 UIPC1 domains for VHPOP and Fast Downward heuristics.

Domain	n	ILP _c	ADDR	FD-Add	FD-FF
Bottleneck	10	1	10	10	10
CaveDiving	10	0	0	6	6
ChessboardPebbling	10	0	0	10	6
Pegsol	10	0	0	10	10
PegsolRow5	10	1	3	5	5
SlidingTiles	10	0	0	10	10
Total	60	2	13	51	47

Due to a multiple order of magnitude difference in runtime, it is clear why ILP_c being more informed does not pay out, as the runtime cost to evaluate the heuristic is far too high. Significant efficiency improvements to our ILP model would be required for it to be a viable alternative to other POCL heuristics. Breakdowns of runtime for each instance are provided in Table 5.3 through to Table 5.8. These show that while state-based heuristics in Fast Downward typically outperform ADDR in VHPOP, there are some instances where ADDR is fastest; although most of these cases can be attributed to the translation time from PDDL to SAS+ by Fast Downward.

Figure 5.2 shows a log scaled comparison of the number of nodes generated by ILP_c and ADDR on instances in IPC3 which they both solved before timeout. It is clear that ILP_c is a more informed heuristic, as it generated less nodes on all instances. However, the difference in nodes generated was not enough to overcome the runtime weaknesses of our heuristic. Despite this, a clear and significant difference in generated nodes is an important and promising result. Without counting relaxations this difference would likely be more pronounced, and so a more efficiently modelled delete-relaxation heuristic could provide enough of an information benefit that heuristic runtime could be offset by a smaller search tree. Breakdowns of generated nodes for each instance are provided in Table 5.9 through to Table 5.14. These tables show that state-based heuristics usually generated less search nodes than POCL heuristics. In most circumstances this would

Table 5.3: Solve time in seconds for the first 10 instances of the Depots domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
Depots1	151.37	0.01	0.65	0.64
Depots2	-	0.3	4.94	4.81
Depots3	-	-	4.38	4.57
Depots4	-	-	4.31	4.34
Depots5	-	-	4.59	4.45
Depots6	-	-	-	-
Depots7	-	1.3	4.42	4.32
Depots8	-	-	8.07	4.57
Depots9	-	-	-	156.65
Depots10	-	0.86	4.88	4.64

Table 5.4: Solve time in seconds for the first 10 instances of the DriverLog domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
DriverLog1	14.07	0.00	0.08	0.09
DriverLog2	-	0.04	0.12	0.10
DriverLog3	54.20	0.01	0.11	0.10
DriverLog4	-	0.17	0.13	0.13
DriverLog5	-	0.01	0.10	0.13
DriverLog6	142.48	0.01	0.12	0.12
DriverLog7	90.78	0.01	0.12	0.13
DriverLog8	-	0.03	0.14	0.14
DriverLog9	-	0.01	0.18	0.16
DriverLog10	-	0.01	0.20	0.20

Table 5.5: Solve time in seconds for the first 10 instances of the FreeCell domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
FreeCell1	-	0.03	5.26	5.27
FreeCell2	-	-	6.82	6.57
FreeCell3	-	-	7.63	7.69
FreeCell4	-	-	8.14	8.11
FreeCell5	-	-	10.04	9.93
FreeCell6	-	-	12.37	12.41
FreeCell7	-	-	12.44	12.28
FreeCell8	-	-	15.44	14.81
FreeCell9	-	-	15.5	14.73
FreeCell10	-	-	18.58	18.2

5 Evaluation

Table 5.6: Solve time in seconds for the first 10 instances of the Rovers domain from IPC3. A dash represents a timeout.

Instance	ILP_c	ADDR	FD-Add	FD-FF
Rovers1	19.2	0.04	0.09	0.09
Rovers2	1.82	0	0.08	0.07
Rovers3	12.15	0	0.09	0.1
Rovers4	3.78	0	0.08	0.1
Rovers5	-	-	0.11	0.11
Rovers6	-	-	0.11	0.1
Rovers7	-	0.04	0.1	0.11
Rovers8	-	1.56	0.14	0.12
Rovers9	-	4.12	0.13	0.11
Rovers10	-	-	0.15	0.15

Table 5.7: Solve time in seconds for the first 10 instances of the Satellite domain from IPC3. A dash represents a timeout.

Instance	ILP_c	ADDR	FD-Add	FD-FF
Satellite1	14.22	0.01	0.09	0.09
Satellite2	181.54	0.00	0.08	0.11
Satellite3	95.12	0.01	0.10	0.09
Satellite4	-	9.32	0.11	0.12
Satellite5	-	0.01	0.12	0.12
Satellite6	-	0.01	0.14	0.13
Satellite7	-	8.29	0.16	0.15
Satellite8	-	0.01	0.22	0.22
Satellite9	-	0.01	0.24	0.25
Satellite10	-	0.01	0.30	0.30

Table 5.8: Solve time in seconds for the first 10 instances of the ZenoTravel domain from IPC3. A dash represents a timeout.

Instance	ILP_c	ADDR	FD-Add	FD-FF
ZenoTravel1	6.07	0.00	0.10	0.10
ZenoTravel2	-	0.03	0.08	0.09
ZenoTravel3	169.88	0.01	0.13	0.12
ZenoTravel4	-	0.76	0.13	0.13
ZenoTravel5	-	1.20	0.15	0.17
ZenoTravel6	-	0.02	0.15	0.15
ZenoTravel7	-	0.74	0.17	0.17
ZenoTravel8	-	0.97	0.30	0.28
ZenoTravel9	-	1.06	0.28	0.29
ZenoTravel10	-	5.46	0.29	0.28

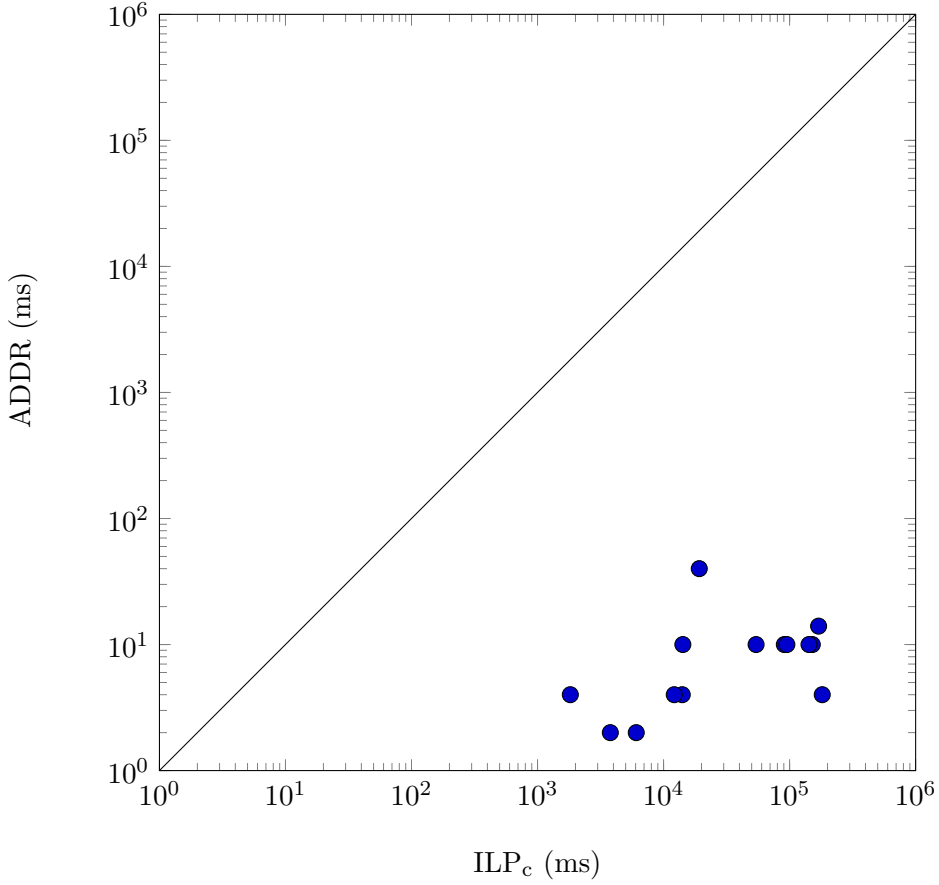


Figure 5.1: Runtime comparison between ILP_c and ADDR on IPC3 domains, log scale.

not be the case, but VHPOP is operating on propositional PDDL and fast downward translates that propositional PDDL into SAS+ before solving. SAS+ allows for multi-values variables and other more expressive techniques which give it an advantage over propositional PDDL. Hence, the result that state-based methods generate less nodes is not evidence that state-based planning creates smaller search trees, as the opposite is true, but instead that SAS+ generates smaller search trees than propositional PDDL.

Overall, these results indicate that the use of ILP-based heuristics for finding unsolvable problems is not promising, and a modified approach would be required to better exploit potential advantages of ILP-based heuristics for unsolvable domains. However, there are no conclusive results that rule out our claims about the potential benefits of NP-hard POCL heuristics on unsolvable domains. Additionally, it is clear that our ILP_c heuristic is more informed than the ADDR heuristic, and this advantage in reducing the size of the search tree provides evidence that more runtime efficient variations of our heuristics could be effective.

5 Evaluation

Table 5.9: Number of nodes generated for the first 10 instances of the Depots domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
Depots1	651	4721	83	136
Depots2	-	289521	195	382
Depots3	-	-	796	13438
Depots4	-	-	3975	84177
Depots5	-	-	17443	54639
Depots6	-	-	-	-
Depots7	-	1125861	3788	14310
Depots8	-	-	700926	46652
Depots9	-	-	-	28081239
Depots10	-	901451	89359	51639

Table 5.10: Number of nodes generated for the first 10 instances of the DriverLog domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
DriverLog1	271	771	61	61
DriverLog2	-	53231	32945	23803
DriverLog3	1141	20311	178	185
DriverLog4	-	300611	43202	32478
DriverLog5	-	6181	565	33976
DriverLog6	2011	6261	6404	7360
DriverLog7	1501	3611	375	486
DriverLog8	-	26341	14914	71203
DriverLog9	-	4171	18164	3780
DriverLog10	-	21711	856	564

Table 5.11: Number of nodes generated for the first 10 instances of the FreeCell domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
FreeCell1	-	28901	52	57
FreeCell2	-	-	136	100
FreeCell3	-	-	145	148
FreeCell4	-	-	1130	350
FreeCell5	-	-	1434	337
FreeCell6	-	-	2102	979
FreeCell7	-	-	1228	5272
FreeCell8	-	-	3244	1380
FreeCell9	-	-	3547	1308
FreeCell10	-	-	4473	1691

Table 5.12: Number of nodes generated for the first 10 instances of the Rovers domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
Rovers1	2341	150871	98	98
Rovers2	381	1681	68	68
Rovers3	1371	1711	189	136
Rovers4	531	931	67	73
Rovers5	-	-	778	777
Rovers6	-	-	2386	2446
Rovers7	-	90231	587	598
Rovers8	-	2861541	1666	1035
Rovers9	-	6524031	1764	1550
Rovers10	-	-	3624	3208

Table 5.13: Number of nodes generated for the first 10 instances of the Satellite domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
Satellite1	1681	1431	70	84
Satellite2	6861	2881	384	156
Satellite3	4451	1821	208	208
Satellite4	-	22055171	562	622
Satellite5	-	5471	1187	1393
Satellite6	-	6071	819	1152
Satellite7	-	14454561	3663	2183
Satellite8	-	11461	5186	3351
Satellite9	-	13891	3764	5414
Satellite10	-	15371	5256	7643

Table 5.14: Number of nodes generated for the first 10 instances of the ZenoTravel domain from IPC3. A dash represents a timeout.

Instance	ILP _c	ADDR	FD-Add	FD-FF
ZenoTravel1	141	731	7	7
ZenoTravel2	-	130831	35	46
ZenoTravel3	3851	9431	174	184
ZenoTravel4	-	2457891	128	264
ZenoTravel5	-	3348611	255	305
ZenoTravel6	-	38241	485	411
ZenoTravel7	-	1870181	249	348
ZenoTravel8	-	2151051	430	1389
ZenoTravel9	-	1366201	1041	1357
ZenoTravel10	-	6992631	1912	2687

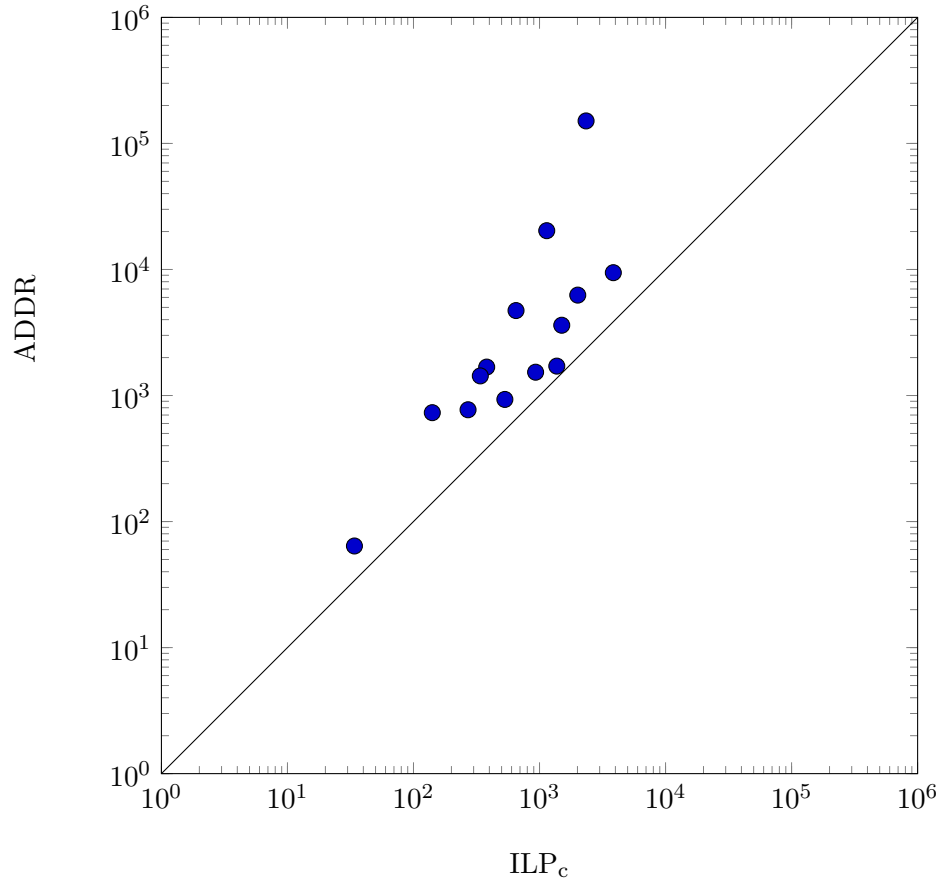


Figure 5.2: Generated nodes comparison between ILP_c and ADDR on IPC3 domains, log scale.

Future Work

While our heuristic is not comparable in effectiveness to the best current POCL heuristics for solvable and unsolvable instances, we have demonstrated that heuristics which account for all POCL search node information can be substantially more informed than heuristics currently used for POCL planning. Hence, avenues of future work to improve the runtime of our heuristic could lead to it outperforming other heuristics.

Firstly, the ILP model used for our heuristic is not efficient. By developing a model from scratch instead of using the model by [Imai and Fukunaga \(2015\)](#) as a guideline, a more simple and effective ILP could be created. For example the *init* and *goal* steps do not have to be considered separately. Satisfaction of goals could be joined with the constraint requiring the satisfaction of preconditions, and the initial propositions would be established as effects of *init*, just like the effects of any other action. Action and step variables and constraints could also be combined, with an additional constraint requiring that all steps must be executed, as otherwise actions and steps are equivalent. In addition, action effect and step effect variables could be removed, as they are currently only used to ensure that propositions have an achiever, but this could be done by directly checking the positive effects of previously executed actions which occur after all possible deletions, in an inverse way to how preconditions are currently verified within our model. Finally, the way that an upper bound *ub* is determined in our model is very relevant to runtime, as it lowers the number of values that time variables can hold, and tightens constraints on the objective value. Currently, the way that *ub* is calculated assumes that the solution of the ILP will be a completely linear plan. However, our ILP model allows for actions to run in parallel if possible, and by accounting for this, a tighter upper bound on time variable values could be determined, improving the runtime of the ILP.

Each calculation of the ILP model for a new search node is done almost entirely inde-

6 Future Work

pendently in our implementation. The only information that is carried from previous calculations is the objective value, for use in determining an upper bound on the new objective value. However, the solution found by the ILP model at any search node will contain large amounts of important information about how search should progress from that node. For example, actions added to the plan by the ILP could be given higher priority for resolving flaws, as they are more likely to be required in a solution plan. The solution to the ILP of a parent search node could also sometimes be used to generate a feasible, but not necessarily optimal, solution for its children. This is a commonly used concept in linear programming called *warmstarting*, which can significantly improve the runtime of ILP calculations.

There are also cases where evaluating our heuristic on a specific search node is not required. For example, in cases where a flaw has been repaired without a step being added, the only possible change in heuristic value from the parent node is if the new node has become unsolvable, which can often be determined by looking at the ILP solution of the parent node. By only using our more expensive heuristic for critical branching points in the search tree, for example adding a new step with delete effects to the plan, and using cheaper evaluations for less critical search nodes, performance could be improved.

The number of action and proposition variables created in our ILP could also be restricted by utilising information already obtained through the search process. For example, actions and propositions which can be found, using planning graphs or other means, to be either unreachable or not useful, do not require associated variables within the ILP. If there is knowledge of how many times a certain action might need to be executed or proposition might need to be achieved, this can also inform how many instance variables are created in the ILP. There might also be information about when a certain action or proposition can occur, for example if an action can at the earliest be executed at time step 4, then its corresponding time variable in the ILP should have its domain restricted accordingly.

Different approaches could also be looked into for future work. For example, further relaxations, such as a linear relaxation of some version of our heuristic, could be promising. Alternatively, further restrictions could be applied, for example evaluating our heuristic for optimal planning, where the output plan must of minimal length. Given that our heuristic is admissible, it could easily be translated to optimal planning, and this could lead to more favourable comparisons with alternative heuristics. By changing the optimisation function of our ILP to minimise makespan instead of the number of actions, future works could also look into the use of our heuristic for minimum makespan planning.

Conclusion

We introduced new heuristics for POCL planning which account for all information within the partial plan they are calculated on. These are the first POCL delete-relaxation heuristics to not ignore information that has already been found by the search process. Our empirical evaluation showed that they have the potential to significantly reduce the size of the search space, at the cost of high runtimes. We then described how to compute these heuristics by using ILP models. While our heuristics weren't competitive with the state of the art in any domains, they were more informed than alternative heuristics for all domains tested. As we created the first prototype of a new class of POCL heuristics which don't relax their search node information, there is a large array of potential future work which can take this theoretical groundwork and build more efficient practical implementations of it. Our report discusses some of these promising lines of future research and how they could improve the effectiveness of ILP-based delete relaxation heuristics for POCL planning, as well as discussing the possible application of our techniques to optimal planning.

Bibliography

- BACKSTROM, C. AND JONSSON, P., 2011. All PSPACE-complete planning problems are equal but some are more equal than others. In *Proceedings of the 4th International Symposium on Combinatorial Search (SOCS 2011)*, 10–17. AAAI Press. doi:10.1609/socs.v2i1.18186. [Cited on page 2.]
- BALCÁZAR, J., 1996. The complexity of searching implicit graphs. *Artificial Intelligence*, 86, 1 (1996), 171–188. doi:10.1016/0004-3702(96)00014-8. [Cited on page 2.]
- BERCHER, P., 2021. A closer look at causal links: Complexity results for delete-relaxation in partial order causal link (POCL) planning. In *Proceedings of the 31st International Conference on Automated Planning and Scheduling (ICAPS 2021)*, 36–45. AAAI Press. doi:10.1609/icaps.v31i1.15944. [Cited on pages 2, 3, 12, and 18.]
- BERCHER, P.; GEIER, T.; AND BIUNDO, S., 2013. Using state-based planning heuristics for partial-order causal-link planning. In *Advances in Artificial Intelligence, Proceedings of the 36th German Conference on Artificial Intelligence (KI 2013)*, 1–12. Springer. doi:10.1007/978-3-642-40942-4_1. [Cited on page 17.]
- BERCHER, P. AND OLZ, C., 2020. POP $\equiv$ POCL, right? Complexity results for partial order (causal link) makespan minimization. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI 2020)*, 9785–9793. AAAI Press. doi:10.1609/aaai.v34i06.6530. [Cited on pages 5 and 8.]
- BLUM, A. L. AND FURST, M. L., 1995. Fast planning through planning graph analysis. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI 1995)*, 1636–1642. Morgan Kaufmann. doi:10.21236/ada303260. [Cited on page 1.]
- BONET, B., 2013. An admissible heuristic for SAS+ planning obtained from the state equation. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI 2013)*, 2268–2274. AAAI Press. doi:10.24963/ijcai.2013/1. [Cited on page 16.]

Bibliography

- BYLANDER, T., 1994. The computational complexity of propositional STRIPS planning. *Artificial Intelligence*, 69, 1-2 (1994), 165–204. doi:10.1016/0004-3702(94)90081-7. [Cited on pages 2, 6, 17, and 18.]
- BYLANDER, T., 1997. A linear programming heuristic for optimal planning. In *Proceedings of the 14th National Conference on Artificial Intelligence (AAAI 1997)*, 694–699. AAAI Press. [Cited on pages 15 and 16.]
- DIMOPOULOS, Y., 2001. Improved integer programming models and heuristic search for AI planning. In *Proceedings of the 6th European Conference on Planning (ECP 2001)*, 301–313. AAAI Press. [Cited on page 16.]
- FIKES, R. E. AND NILSSON, N. J., 1971. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2 (1971), 189–208. doi:10.1016/0004-3702(71)90010-5. [Cited on page 5.]
- FISER, D., 2020. Lifted fact-alternating mutex groups and pruned grounding of classical planning problems. In *Proceedings of the 34th Conference on Artificial Intelligence (AAAI 2020)*, vol. 34, 9835–9842. AAAI Press. doi:10.1609/aaai.v34i06.6536. [Cited on page 34.]
- GEFFNER, H. AND HASLUM, P., 2000. Admissible heuristics for optimal planning. In *Proceedings of the 5th International Conference on Artificial Intelligence Planning Systems (AIPS 2000)*, 140–149. AAAI Press. [Cited on page 34.]
- HELMERT, M., 2006. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26 (2006), 191–246. doi:10.1613/jair.1705. [Cited on pages 1 and 33.]
- HOFFMANN, J. AND NEBEL, B., 2001. The FF planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research*, 14 (2001), 253–302. doi:10.1613/jair.855. [Cited on pages 1, 2, and 34.]
- HÖLLER, D.; BERCHER, P.; AND BEHNKE, G., 2020. Delete- and ordering-relaxation heuristics for htn planning. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI 2020)*, 4076–4083. IJCAI. doi:10.24963/ijcai.2020/564. [Cited on page 17.]
- IMAI, T. AND FUKUNAGA, A., 2015. On a practical, integer-linear programming model for delete-free tasks and its use as a heuristic for cost-optimal planning. *Journal of Artificial Intelligence Research*, 54 (2015), 631–677. doi:10.1613/jair.4936. [Cited on pages 17, 19, and 43.]
- KAMBHAMPATI, S., 1997. Refinement planning as a unifying framework for plan synthesis. *AI magazine*, 18, 2 (1997), 67–98. doi:10.1016/b978-1-4832-1452-8.50126-3. [Cited on page 9.]

- KARLOFF, H., 1991. *Linear Programming*. Birkhäuser. doi:10.1007/978-0-8176-4844-2. [Cited on pages 11 and 12.]
- KARP, R. M., 1972. Reducibility among combinatorial problems. In *Proceedings of a Symposium on the Complexity of Computer Computations*, 85–103. Plenum Press. doi:10.1007/978-1-4684-2001-2_9. [Cited on page 12.]
- KHACHIYAN, L. G., 1979. A polynomial algorithm in linear programming. 244, 5 (1979), 1093–1096. doi:10.1016/0041-5553(80)90061-0. [Cited on page 12.]
- LAWLER, E. L. AND LABETOULLE, J., 1978. On preemptive scheduling of unrelated parallel processors by linear programming. *Journal of the Association for Computing Machinery*, 25, 4 (1978), 612–619. [Cited on page 12.]
- MCDERMOTT, D. V., 1996. A heuristic estimator for means-ends analysis in planning. In *Proceedings of the 3rd International Conference on Artificial Intelligence Planning Systems (AIPS 1996)*, 142–149. AAAI Press. [Cited on page 1.]
- MINTON, S.; BRESINA, J. L.; AND DRUMMOND, M., 1994. Total-order and partial-order planning: A comparative analysis. *Journal of Artificial Intelligence Research*, 2 (1994), 227–262. doi:10.1613/jair.64. [Cited on page 1.]
- MUISE, C.; BECK, J. C.; AND MCILRAITH, S. A., 2016. Optimal partial-order plan relaxation via MaxSAT. *Journal of Artificial Intelligence Research*, 57 (2016), 113–149. doi:10.1613/jair.5128. [Cited on page 9.]
- NGUYEN, X. AND KAMBHAMPATI, S., 2001. Reviving partial order planning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI 2001)*, 459–464. Morgan Kaufmann. [Cited on pages 2 and 17.]
- PENBERTHY, J. S. AND WELD, D. S., 1992. UCPOP: A sound, complete, partial order planner for ADL. In *Proceedings of the 3rd International Conference on Principles of Knowledge Representation and Reasoning (KR 1992)*, 103–114. Morgan Kaufmann. [Cited on page 1.]
- POMMERENING, F.; RÖGER, G.; AND HELMERT, M., 2013. Getting the most out of pattern databases for classical planning. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI 2013)*, 2357–2364. AAAI Press. [Cited on page 16.]
- POMMERENING, F.; RÖGER, G.; HELMERT, M.; AND BONET, B., 2014. LP-based heuristics for cost-optimal planning. In *Proceedings of the 24th International Conference on Automated Planning and Scheduling (ICAPS 2014)*, vol. 24, 226–234. AAAI Press. doi:10.1609/icaps.v24i1.13621. [Cited on pages 16 and 27.]
- TRAUB, J. F. AND WOŹNIAKOWSKI, H., 1982. Complexity of linear programming. *Operations Research Letters*, 1, 2 (1982), 59–62. [Cited on page 12.]

Bibliography

- TURNER, H., 2002. Polynomial-length planning spans the polynomial hierarchy. In *Proceedings of the 8th European Conference on Logics in Artificial Intelligence (JELIA 2002)*, 111–124. Springer. doi:10.1007/3-540-45757-7_10. [Cited on page 15.]
- VAN DEN BRIEL, M.; BENTON, J.; KAMBHAMPATI, S.; AND VOSSEN, T., 2007. An LP-based heuristic for optimal planning. In *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming (CP 2007)*, 651–665. Springer. doi:10.1007/978-3-540-74970-7_46. [Cited on page 16.]
- VAN DEN BRIEL, M. AND KAMBHAMPATI, S., 2005. Optiplan: A planner based on integer programming. *Journal of Artificial Intelligence Research*, 24 (2005), 919–931. doi:10.1613/jair.1698. [Cited on page 16.]
- VIDAL, V. AND GEFFNER, H., 2006. Branching and pruning: An optimal temporal POCL planner based on constraint programming. *Artificial Intelligence*, 170, 3 (2006), 298–335. doi:10.1016/j.artint.2005.08.004. [Cited on page 17.]
- VOSSEN, T.; BALL, M.; LOTEM, A.; AND NAU, D., 1999. On the use of integer programming models in AI planning. In *Proceedings of the 16th International Joint Conference on Artificial Intelligence (IJCAI 1999)*, 304–309. Morgan Kaufmann. [Cited on page 16.]
- WELD, D. S., 1994. An introduction to least commitment planning. *AI Magazine*, 15, 4 (1994), 27–61. [Cited on page 1.]
- WELD, D. S., 2011. Systematic nonlinear planning: A commentary. *AI Magazine*, 32, 1 (2011), 101–103. doi:10.1609/aimag.v32i1.2341. [Cited on page 1.]
- YOUNES, H. L. AND SIMMONS, R. G., 2003. VHPOP: Versatile heuristic partial order planner. *Journal of Artificial Intelligence Research*, 20 (2003), 405–430. doi:10.1613/jair.1136. [Cited on pages 2, 17, and 33.]